CLIENT APPLICATIONS

Real applications on RunShore databases

Four ordinary applications connected to the databases RunShore deploys on local Kubernetes (minikube). Each database is monitored by an observability tool, put under load by the application, and observed before, during and after the load.

VERSION	DATE	ENVIRONMENT	EVIDENCE
1.0	November 2026	Local Kubernetes (minikube)	Captured 2–10 Nov 2026

AT A GLANCE

APPLICATION	DATABASE	OBSERVED WITH
WordPress PHP	**MySQL** Percona XtraDB Cluster, through HAProxy	PMM
IDURAR ERP / CRM Node.js + React	**MongoDB** Percona Server for MongoDB replica set, follows the primary	New Relic
Django Python	**PostgreSQL** CloudNativePG read-write service	PMM
Customer Orders Java Spring Boot	**SAP ASE / Sybase** Read-only application login	Splunk

CONTENTS

1. **WordPress on MySQL**
2. **IDURAR ERP / CRM on MongoDB**
3. **Django on PostgreSQL**
4. **Customer Orders (Spring Boot) on SAP ASE**
5. **What this shows**

All four applications are started, stopped and checked with one script; their database connections are re-established automatically when database pods restart.

```
bash tools/client-apps/runshore_apps.sh start|stop|status  [wordpress|idurar|djan
go|sybasejava|all]
```

Browser address bars, account identifiers, private addresses and usernames have been removed or blurred from all screenshots.

01 WordPress on MySQL

PHP · MySQL (Percona XtraDB Cluster) · monitored with PMM

APPLICATION	DATABASE	CONNECTION	OBSERVABILITY	LOAD
WordPress (PHP)	MySQL · Percona XtraDB Cluster	HAProxy via port-forward 127.0.0.1:3307	PMM	tools/client-apps/wordpress/workload.sh

1.1 Connect to the RunShore MySQL cluster

A port-forward makes the cluster reachable on `127.0.0.1:3307`. MySQL Workbench confirms the connection.

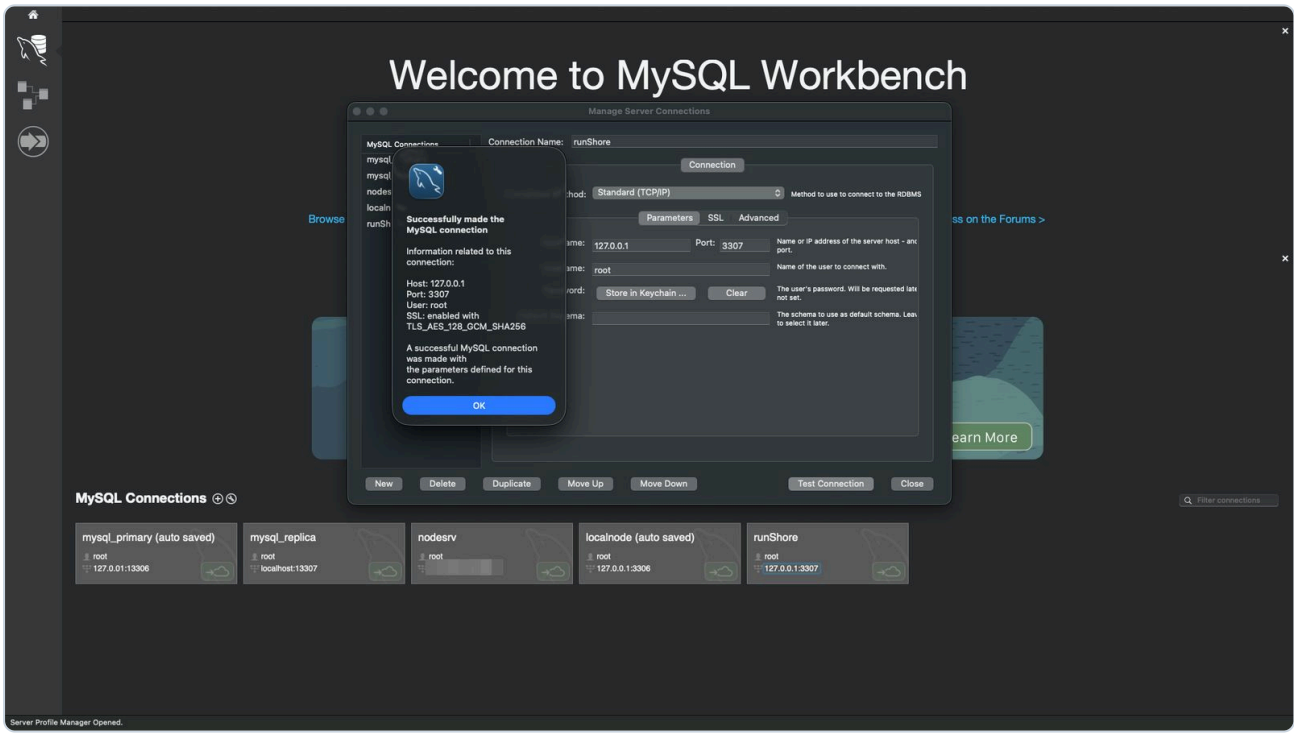


Figure 1.1 MySQL Workbench: successful connection to the RunShore MySQL cluster

1.2 Create the WordPress database and user

A dedicated database and user for WordPress. Use your own strong password; never one from a document.

```
CREATE DATABASE wordpress_demo
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'wordpress_demo'@'%'
  IDENTIFIED BY 'REPLACE_WITH_YOUR_OWN_STRONG_PASSWORD';
```

```
GRANT ALL PRIVILEGES ON wordpress_demo.* TO 'wordpress_demo'@'%';
```

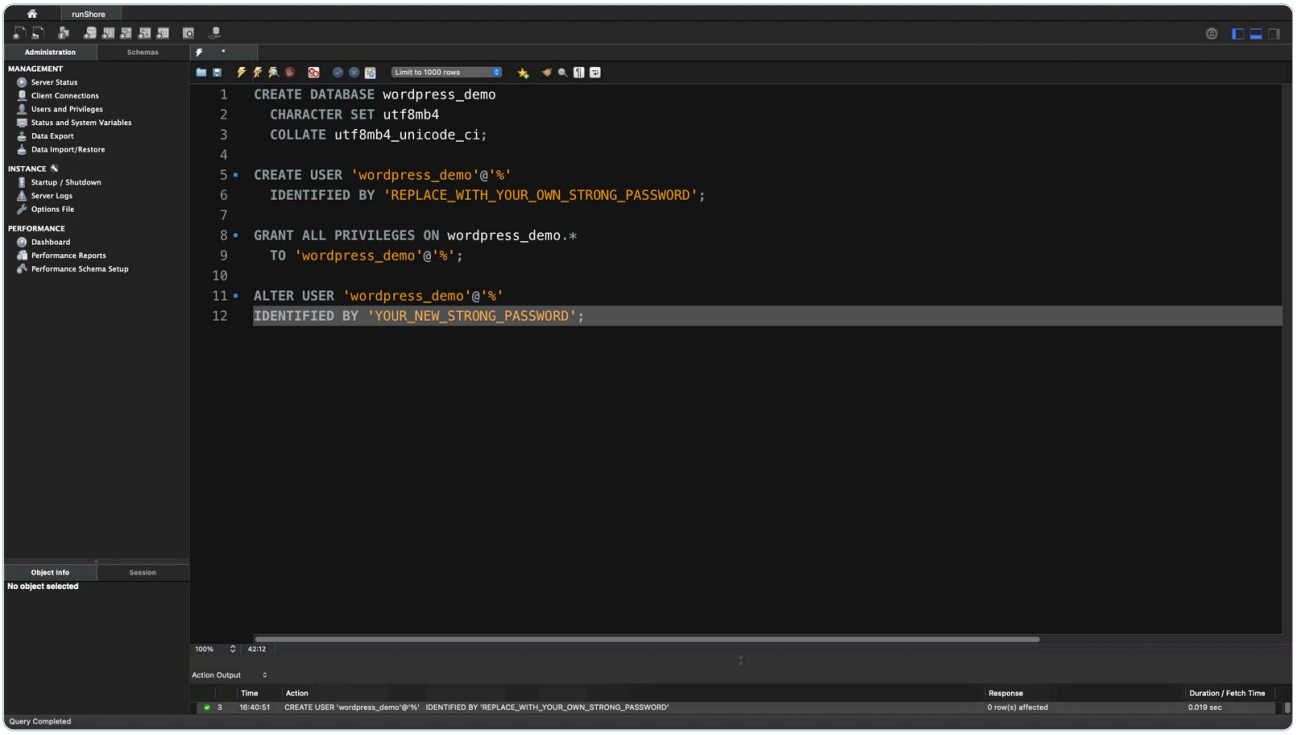


Figure 1.2 The statements run in MySQL Workbench

1.3 Start WordPress

WordPress runs on PHP's built-in server at `127.0.0.1:8088`, alongside the MySQL port-forward on `3307`.



Figure 1.3 Both sessions must stay running

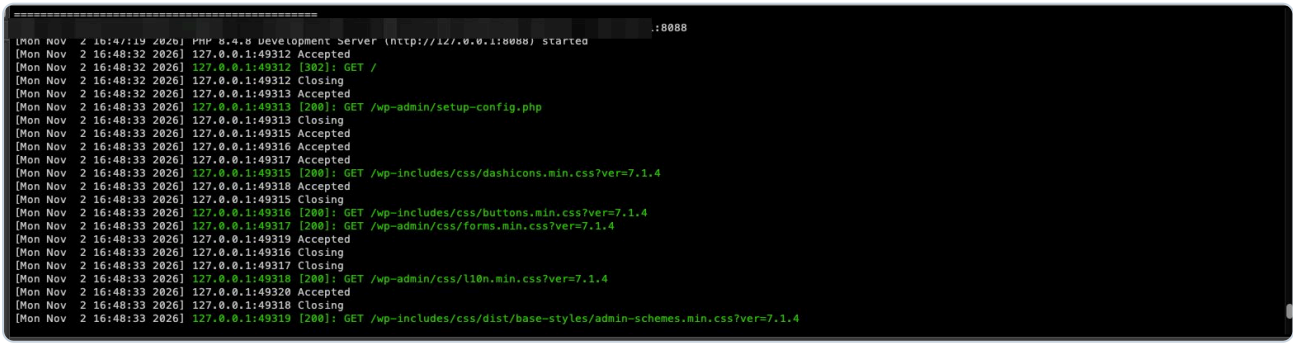


Figure 1.4 PHP server log: WordPress serving its pages

1.4 WordPress is running on RunShore's MySQL

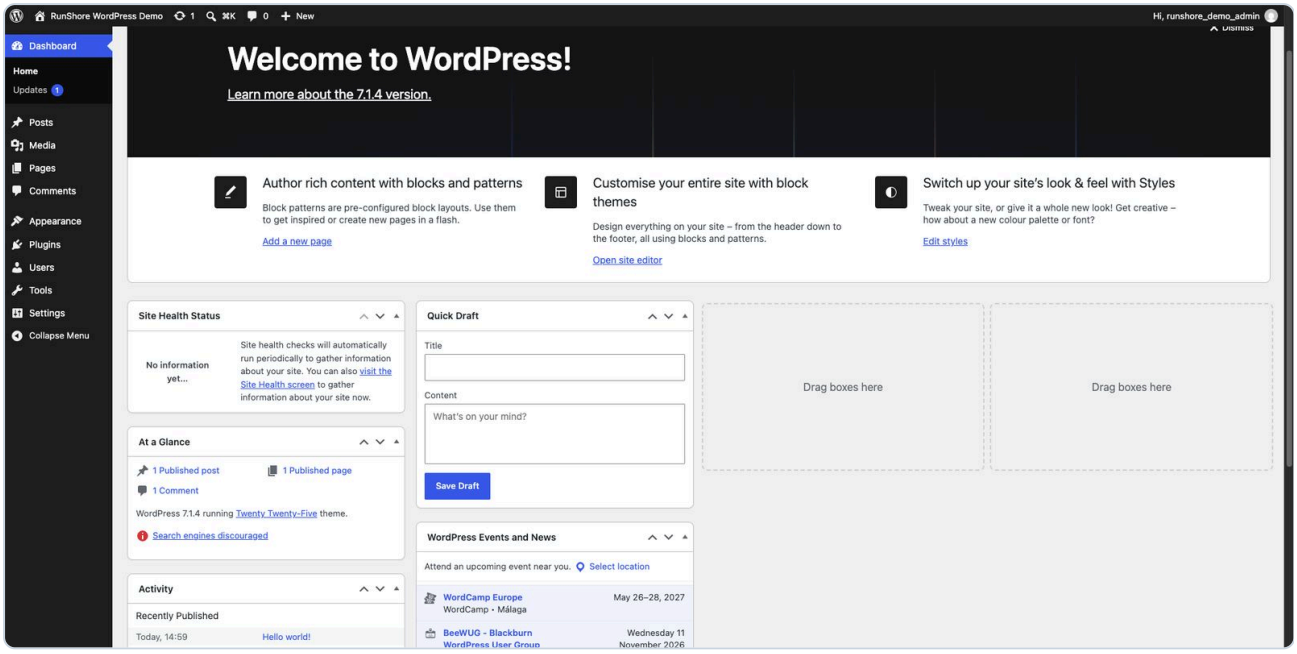


Figure 1.5 WordPress dashboard

1.5 Generate load

```
tools/client-apps/wordpress/workload.sh start
```

The workload writes realistic content through WordPress itself: order posts in a **RunShore Workload** category.

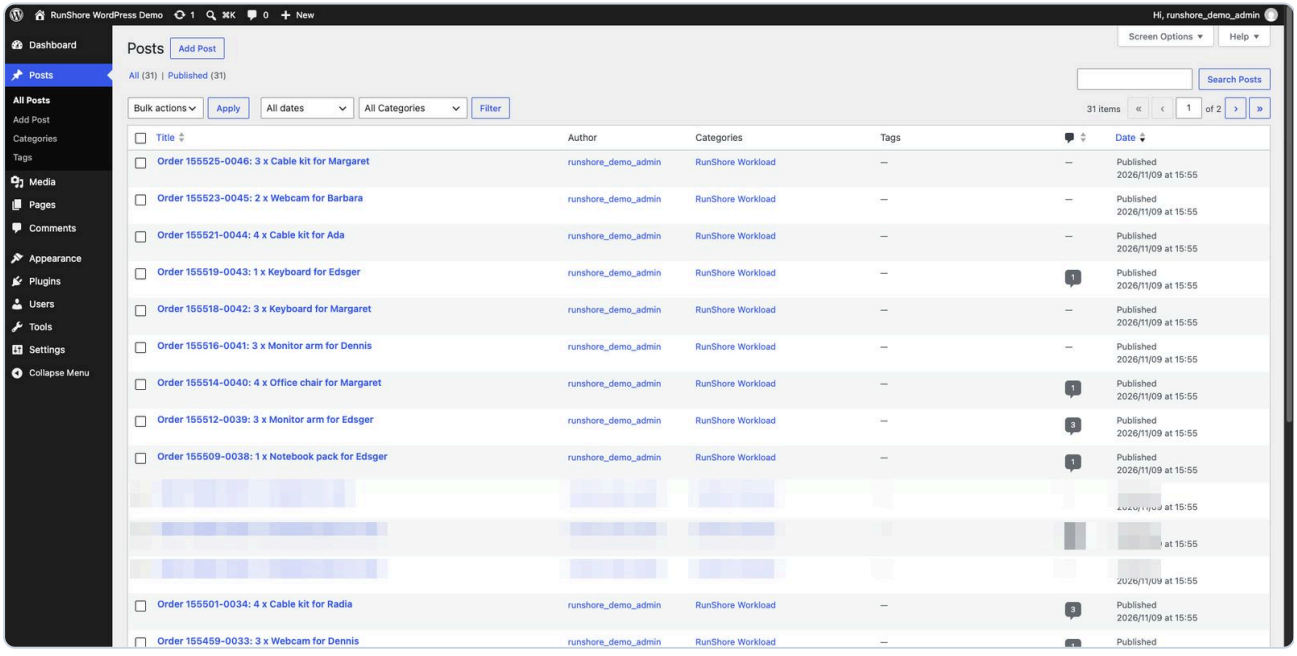


Figure 1.6 Posts created by the workload

1.6 Observe in PMM

Baseline: about 22 operations/s at 0.3 ms latency, no errors, 9 connections.



Figure 1.7 PMM Workload – MySQL: baseline

Additional load: throughput and latency rise; CPU and memory follow.



Figure 1.8 PMM Workload – MySQL: under additional load

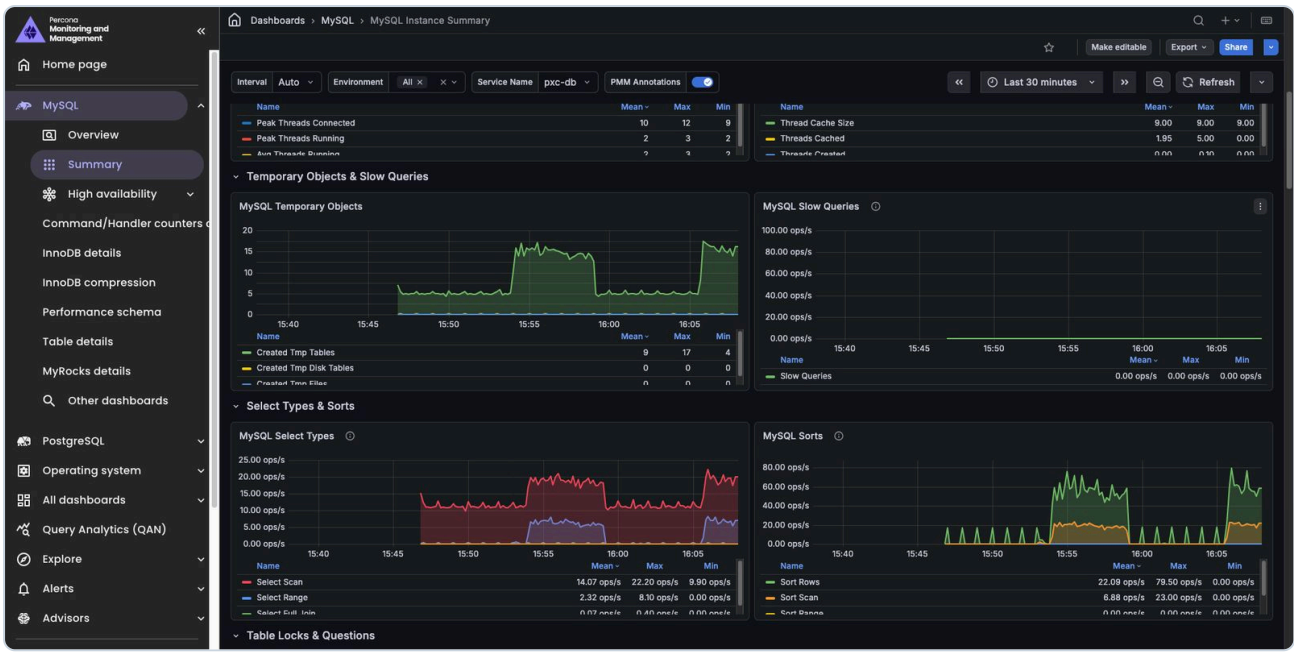


Figure 1.9 PMM MySQL Instance Summary: temporary objects, select types and sorts

OUTCOME

A real application-to-database connection, using RunShore's deployed MySQL infrastructure.

02

IDURAR ERP / CRM on MongoDB

Node.js + React · MongoDB (Percona Server for MongoDB) · monitored with New Relic

APPLICATION	DATABASE	CONNECTION	OBSERVABILITY	LOAD
IDURAR ERP / CRM (Node.js, React)	MongoDB · Percona Server for MongoDB	Replica-set primary, follows failover	New Relic	tools/maintenance/idurar_crm_erp_load.js

2.1 Start IDURAR

The backend (Express, port 8888) and the frontend (Vite, port 3000) connect to the RunShore MongoDB primary.

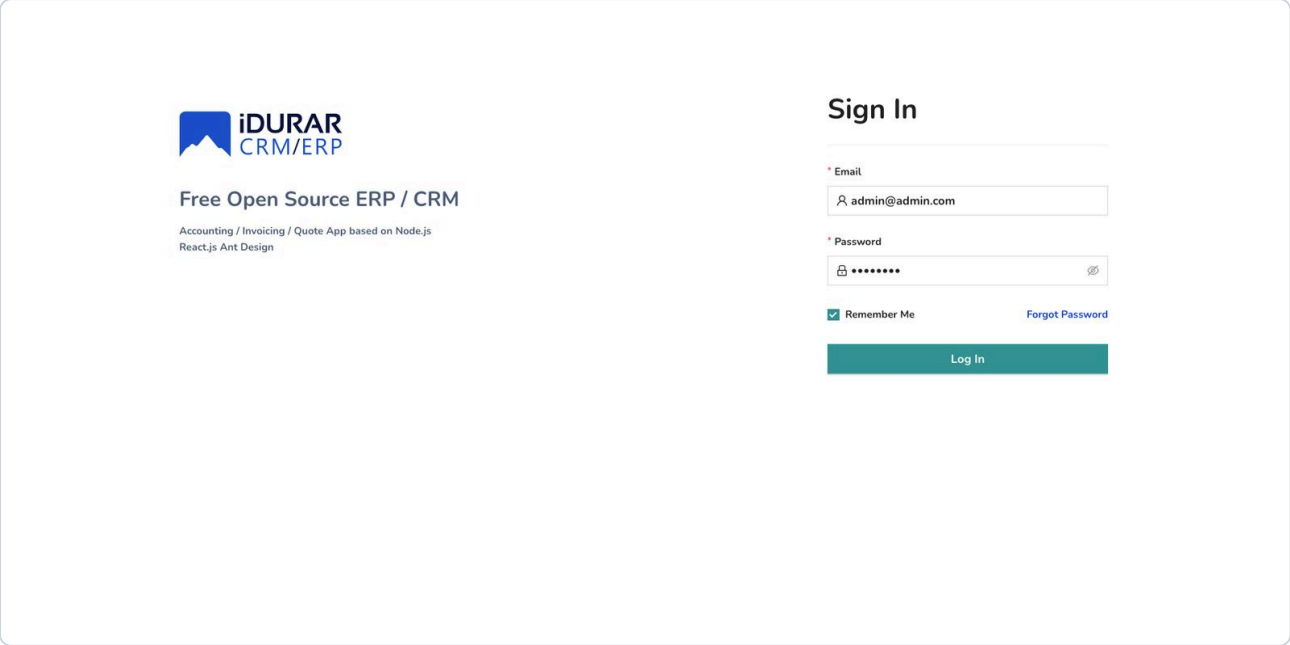


Figure 2.1 IDURAR sign-in

```
$ npm run dev

> idurar-erp-crm@4.1.0 dev
> nodemon src/server.js --ignore public/

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
Express running → On PORT : 8888
```

Figure 2.2 Backend: Express on port 8888

```
$ npm run dev

> idurar-erp-crm@4.1.0 dev
> vite


  VITE v5.4.8  ready in 1305 ms

  ➜  Local:   http://localhost:3000/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Figure 2.3 Frontend: Vite on port 3000

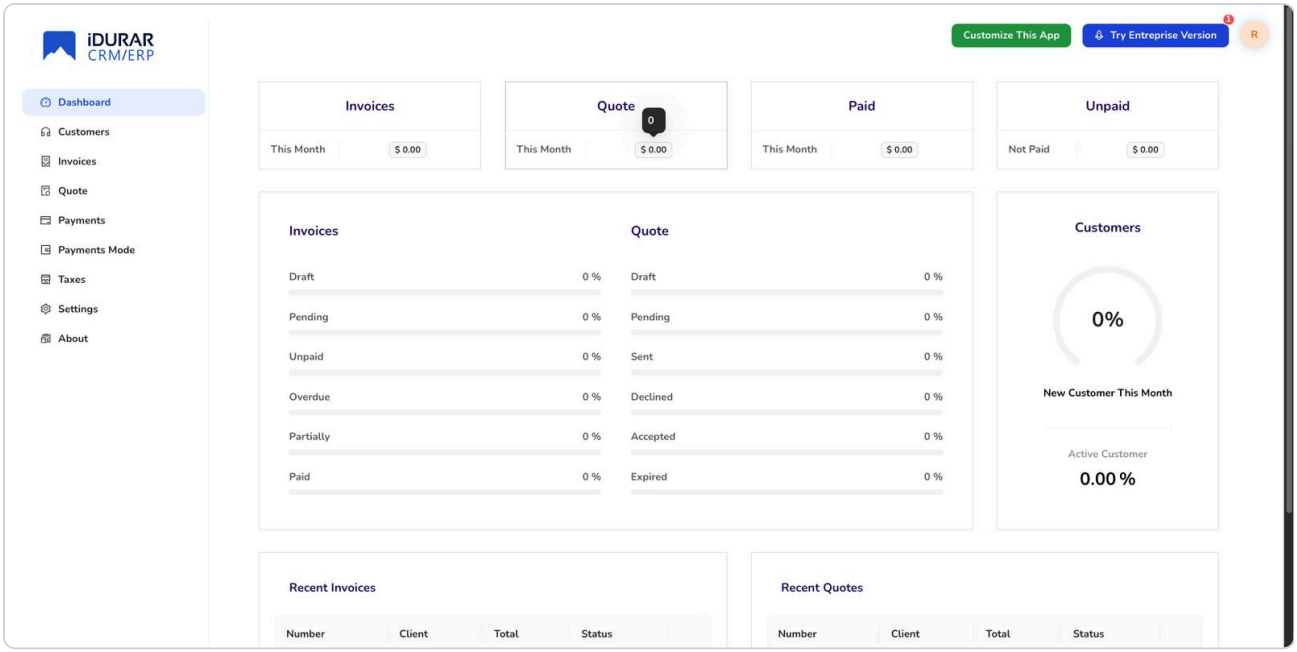


Figure 2.4 IDURAR dashboard before any data

2.2 Register MongoDB with New Relic from RunShore

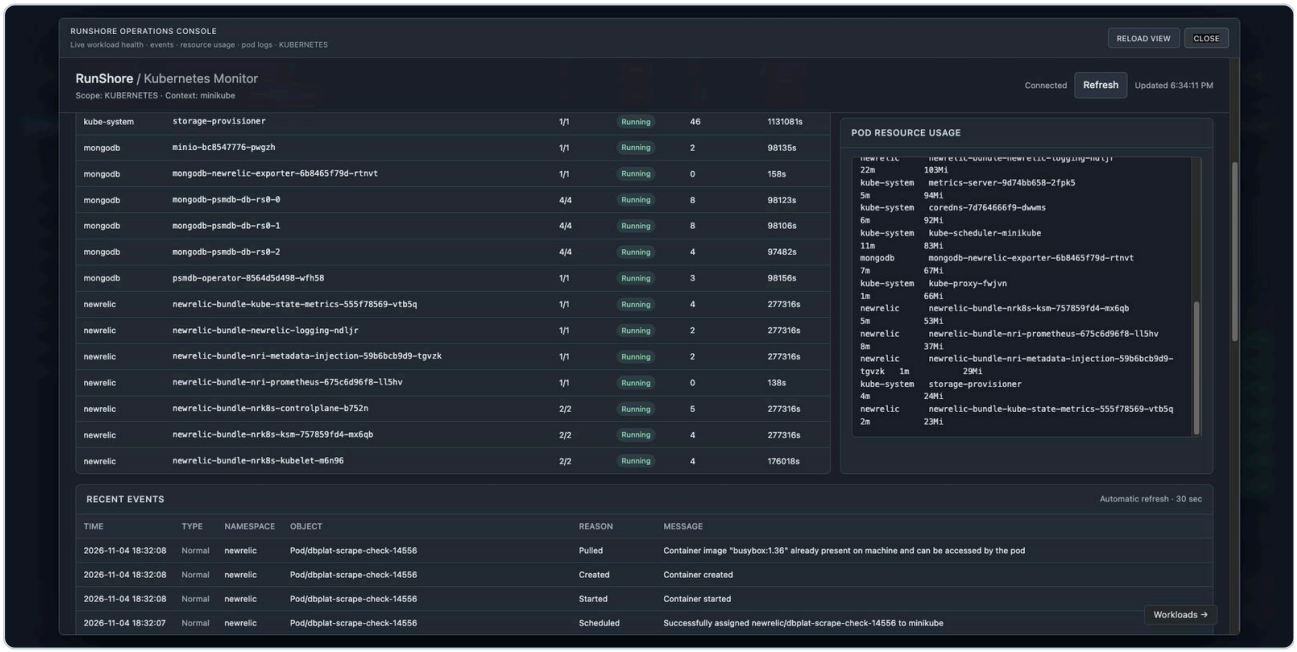


Figure 2.5 RunShore Operations Console: New Relic and MongoDB pods running

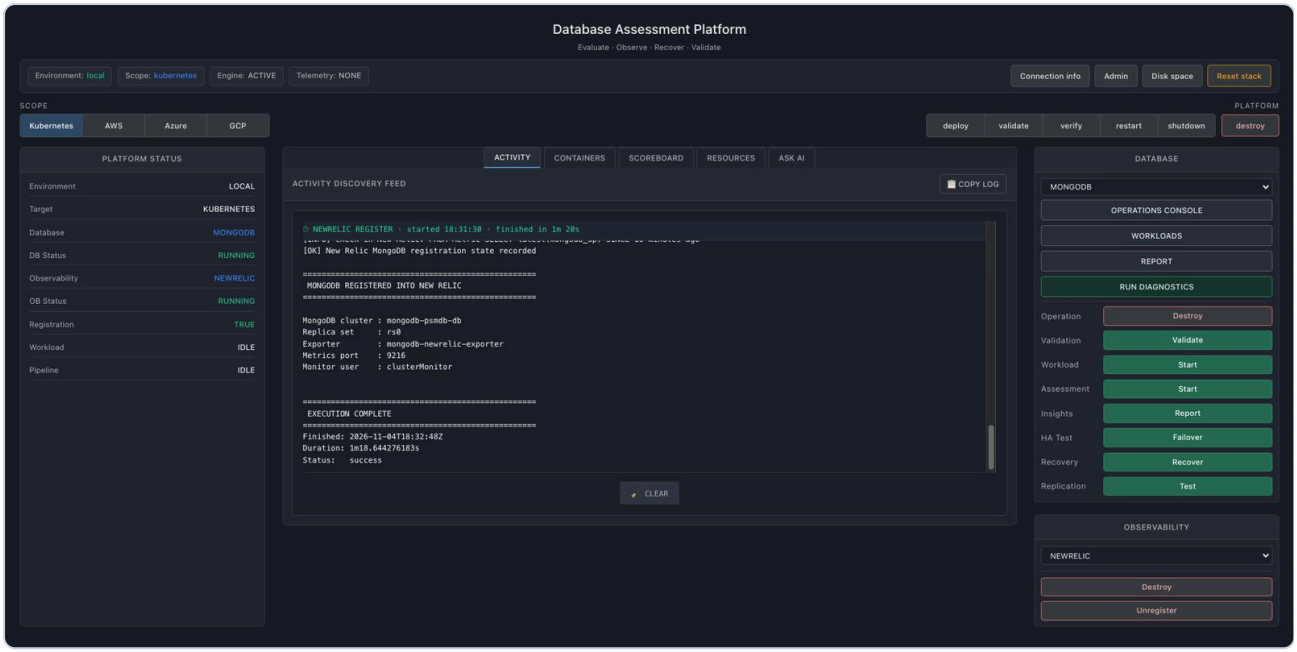


Figure 2.6 RunShore Activity feed: New Relic registration complete

2.3 New Relic baseline

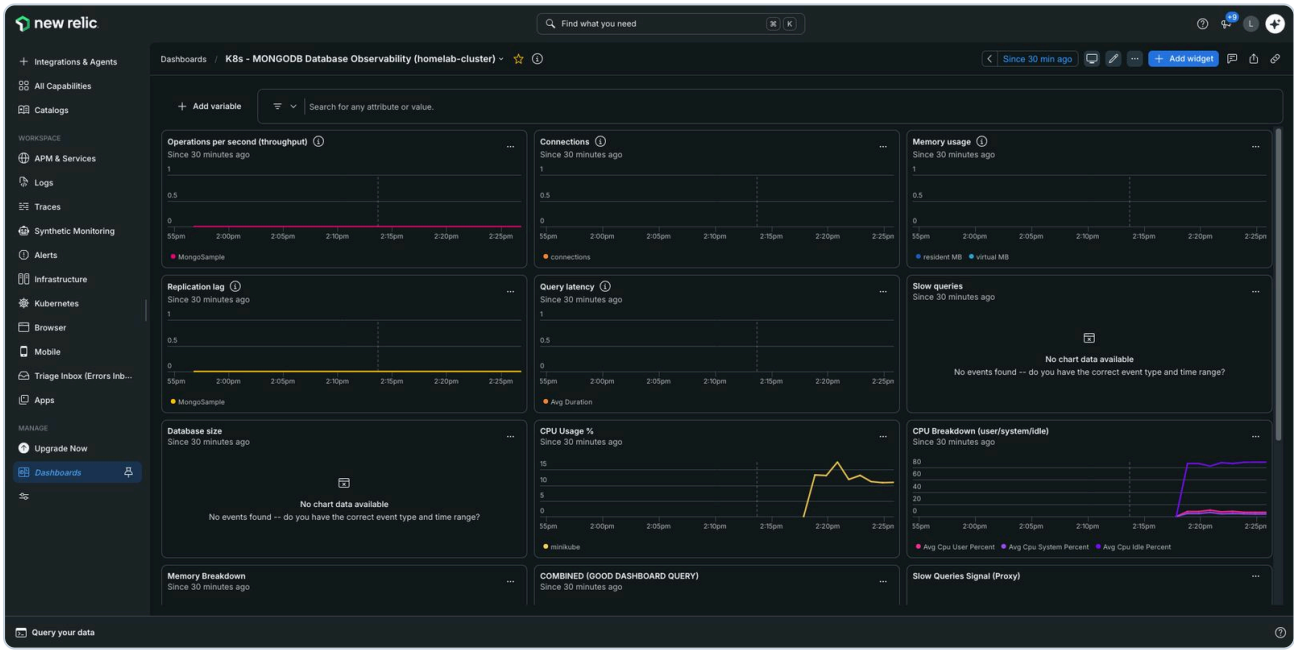


Figure 2.7 New Relic: K8s MongoDB Database Observability

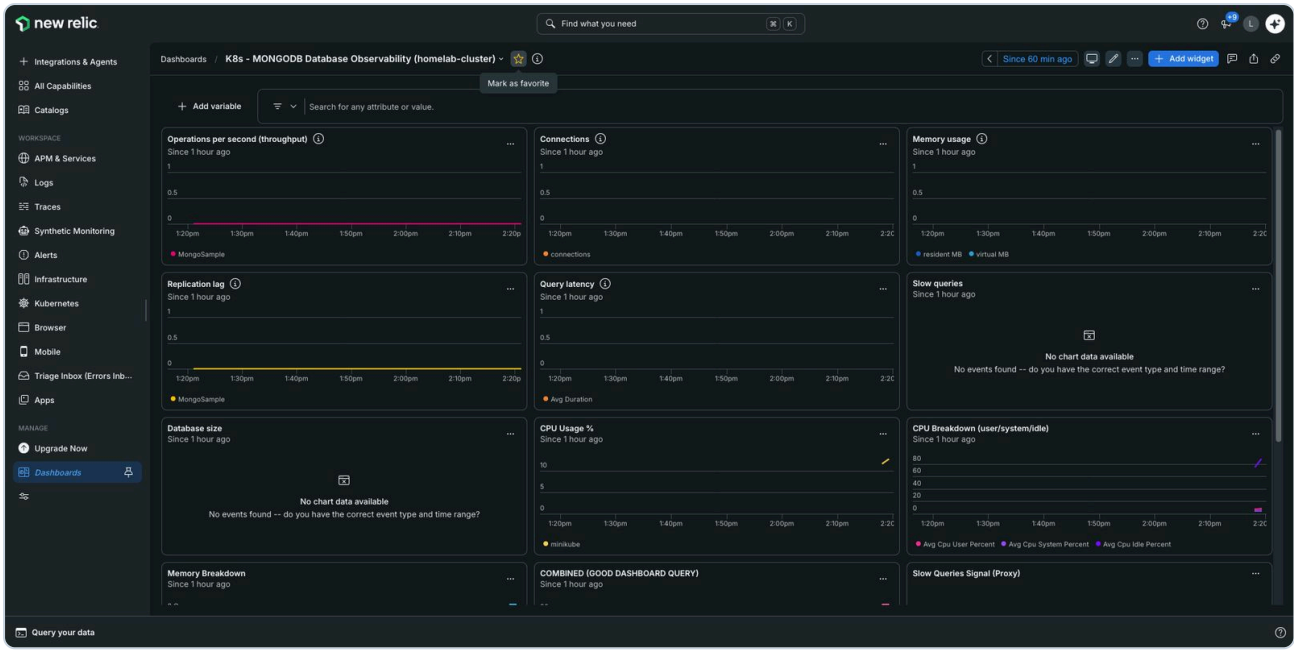


Figure 2.8 Same dashboard, longer time window

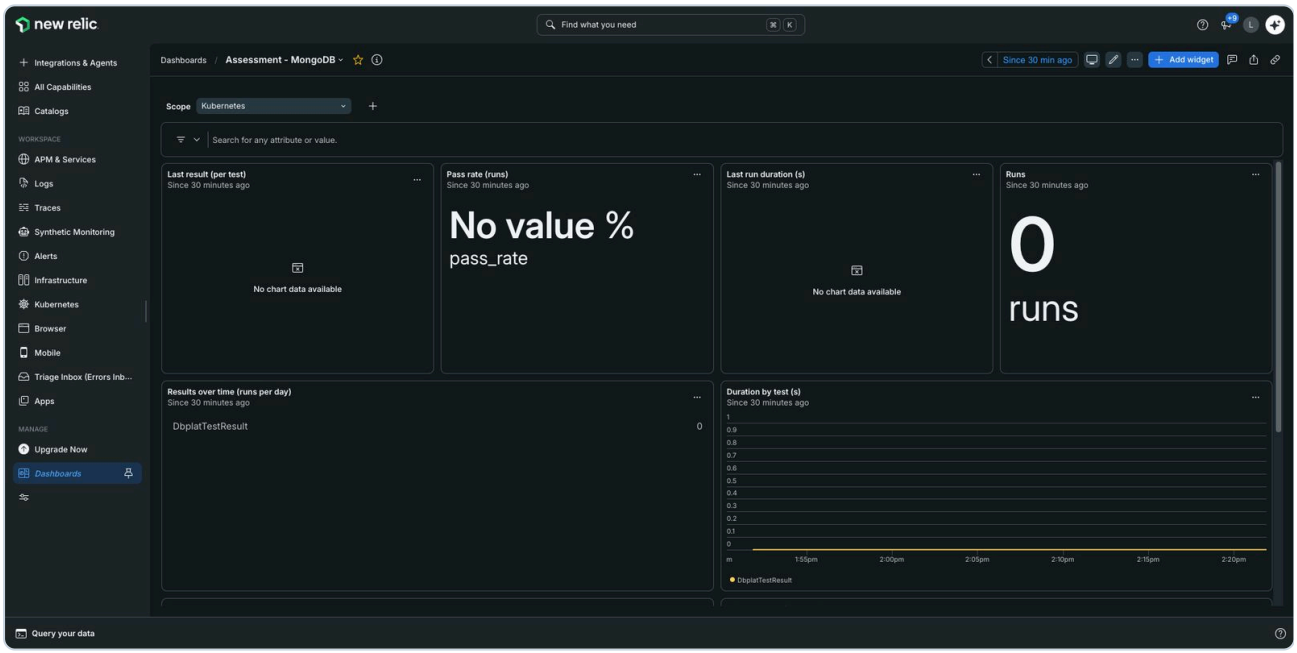


Figure 2.9 New Relic Assessment – MongoDB: no runs yet

2.4 Smoke test of the load generator

```
node tools/maintenance/idurar_crm_erp_load.js --execute --smoke-test
```

- One business bundle: a customer, a quote, an invoice and a payment, created through IDURAR's own API, then deleted: **4 inserts + 4 deletes**.
- Only tagged RunShore test records (`RSLOAD-IDURAR-*`) are touched; earlier test records are purged first, so every run starts clean.
- Runs only after typing `RUNSHORE IDURAR LOAD TEST`; the administrator password is asked for and never stored.

- Result: authentication PASS, bundle created 1/1 (100%) in 0.2 seconds.

2.5 Workload, assessment and the IDURAR load

14:30 MongoDB workload begins

14:35–14:37 assessment

14:40 IDURAR ERP/CRM load begins

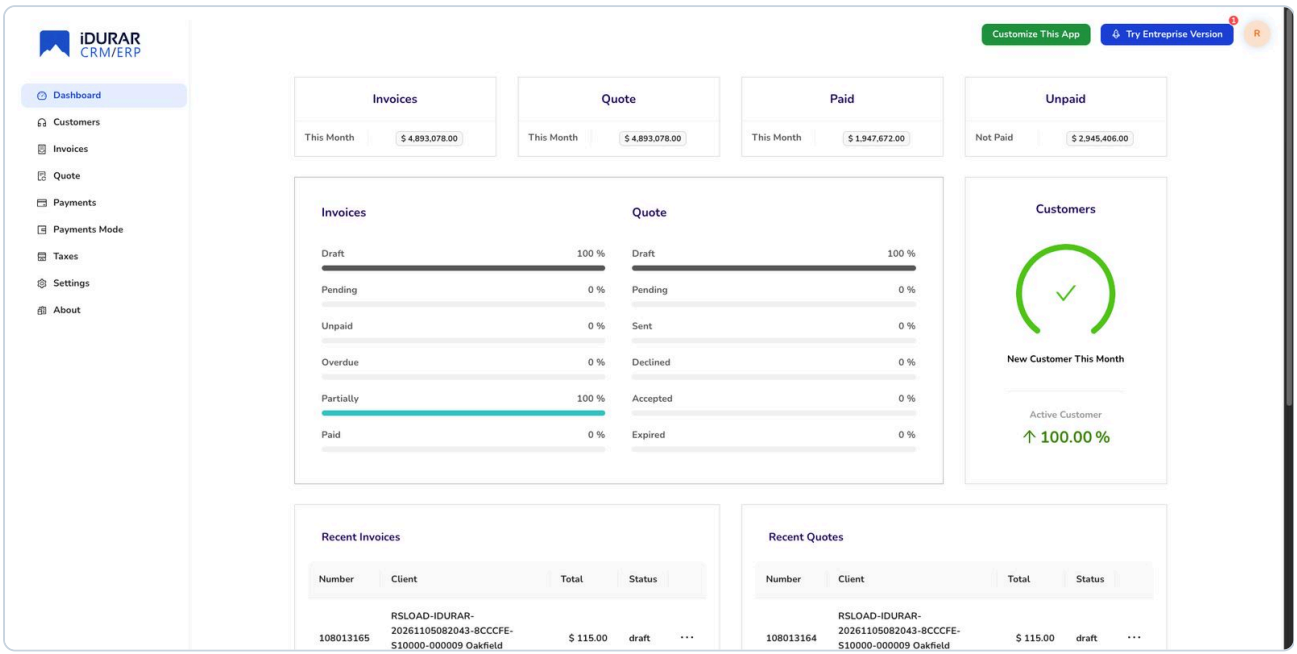


Figure 2.10 IDURAR dashboard after the load: invoices and quotes

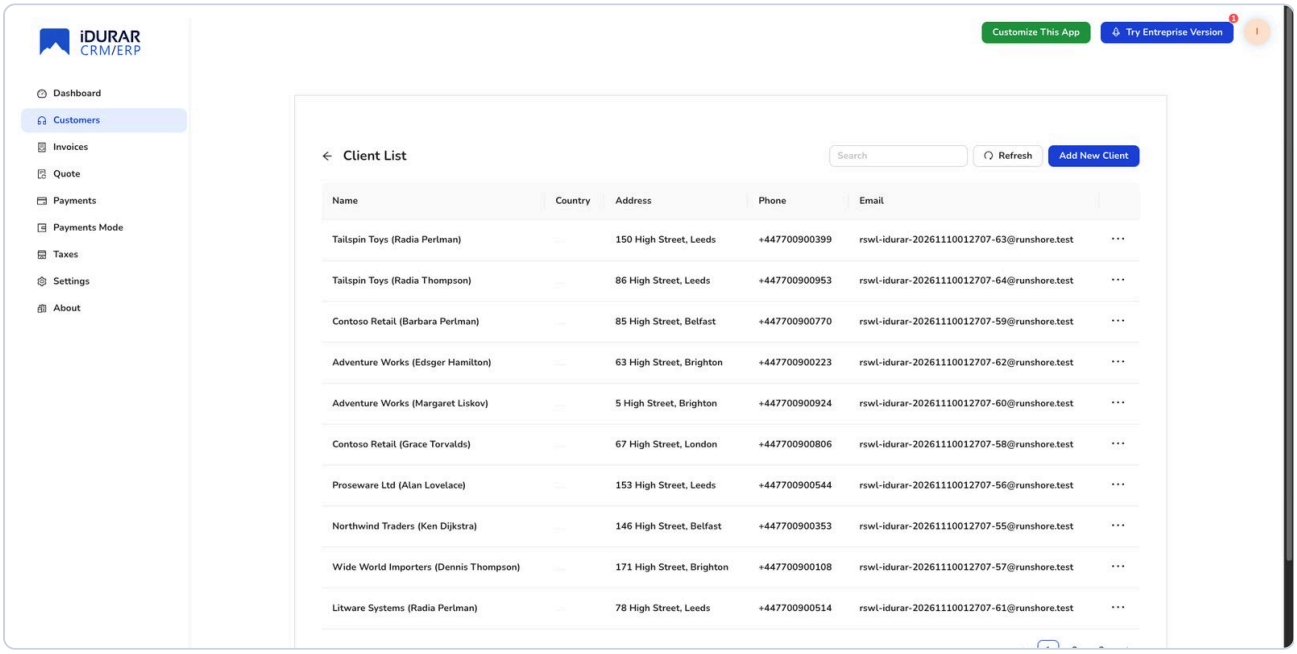


Figure 2.11 IDURAR client list: tagged RunShore test customers

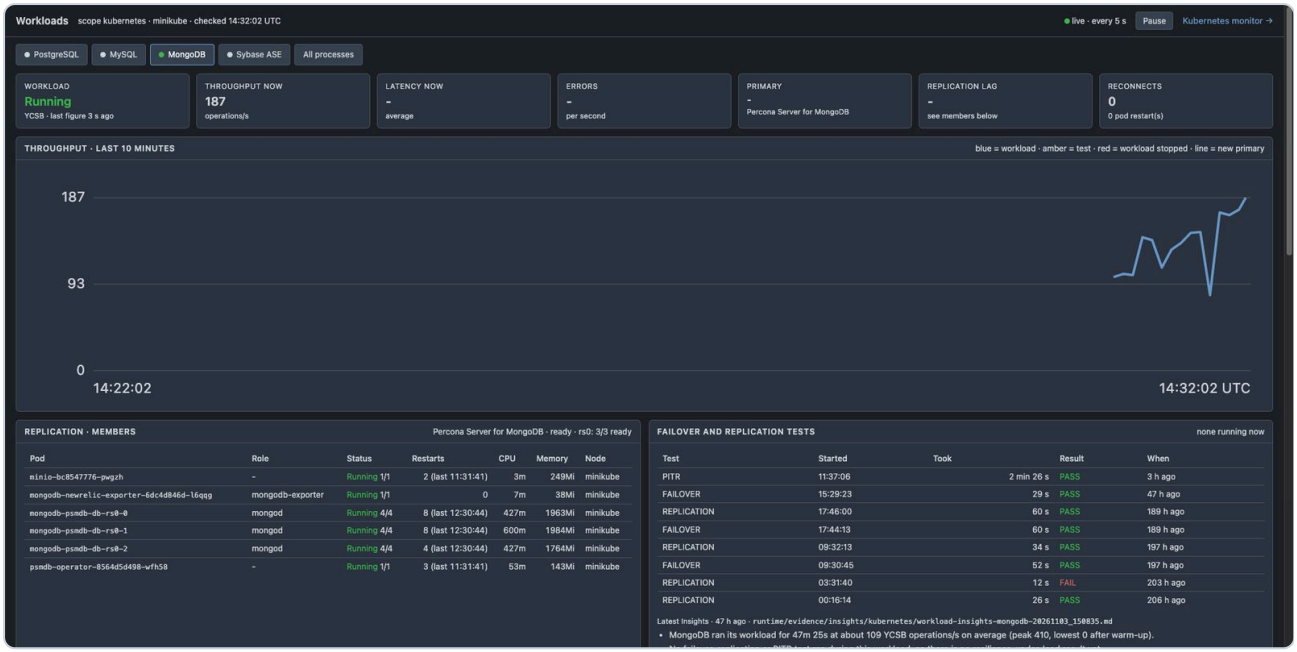


Figure 2.12 RunShore Workloads window: MongoDB workload running, failover and replication tests passing

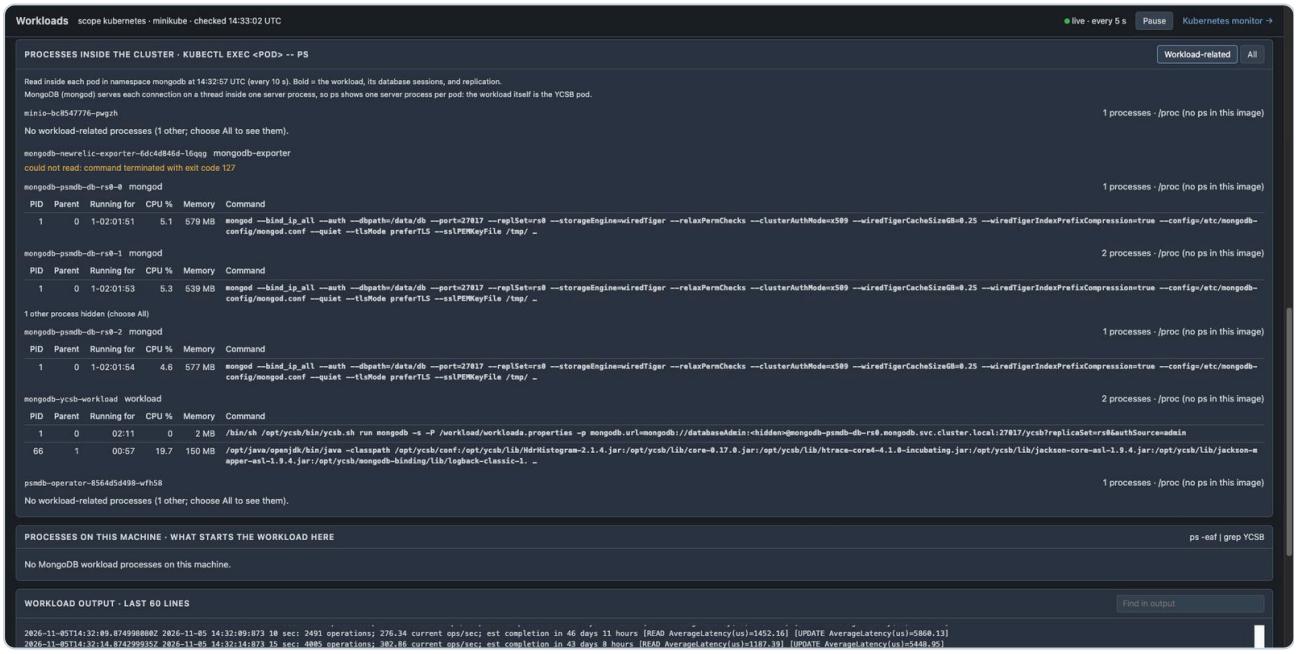


Figure 2.13 Processes inside the cluster during the workload

2.6 New Relic during the workload

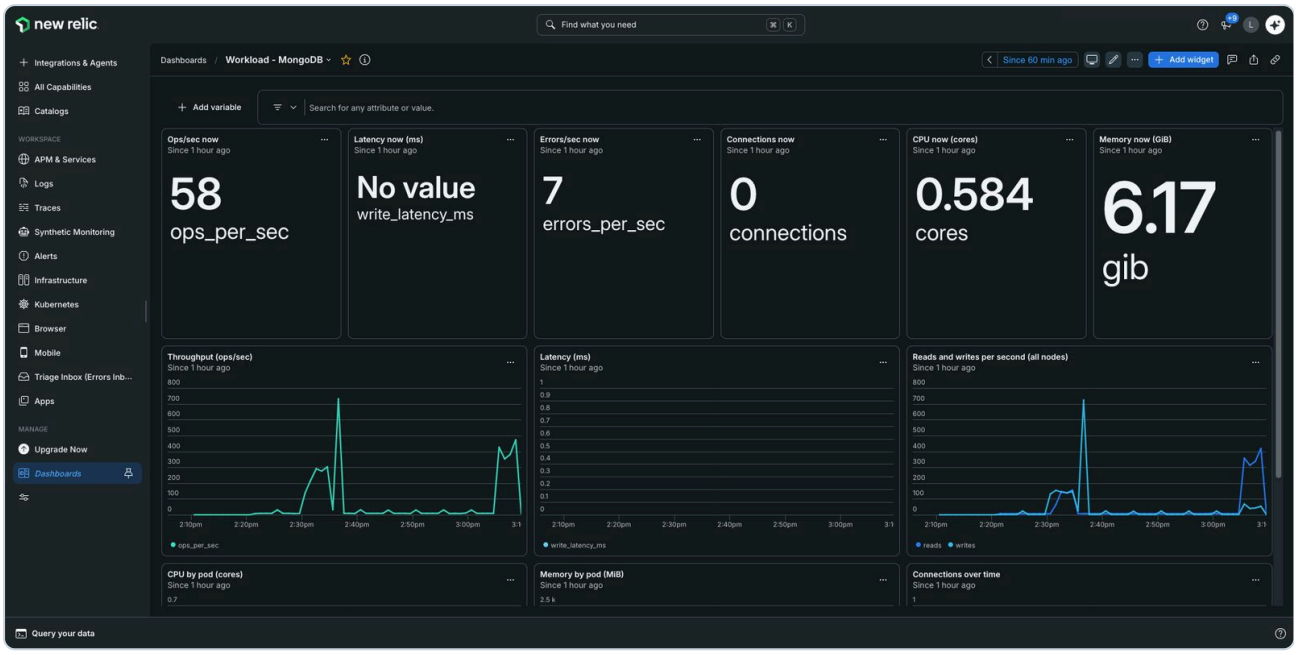


Figure 2.14 New Relic Workload – MongoDB

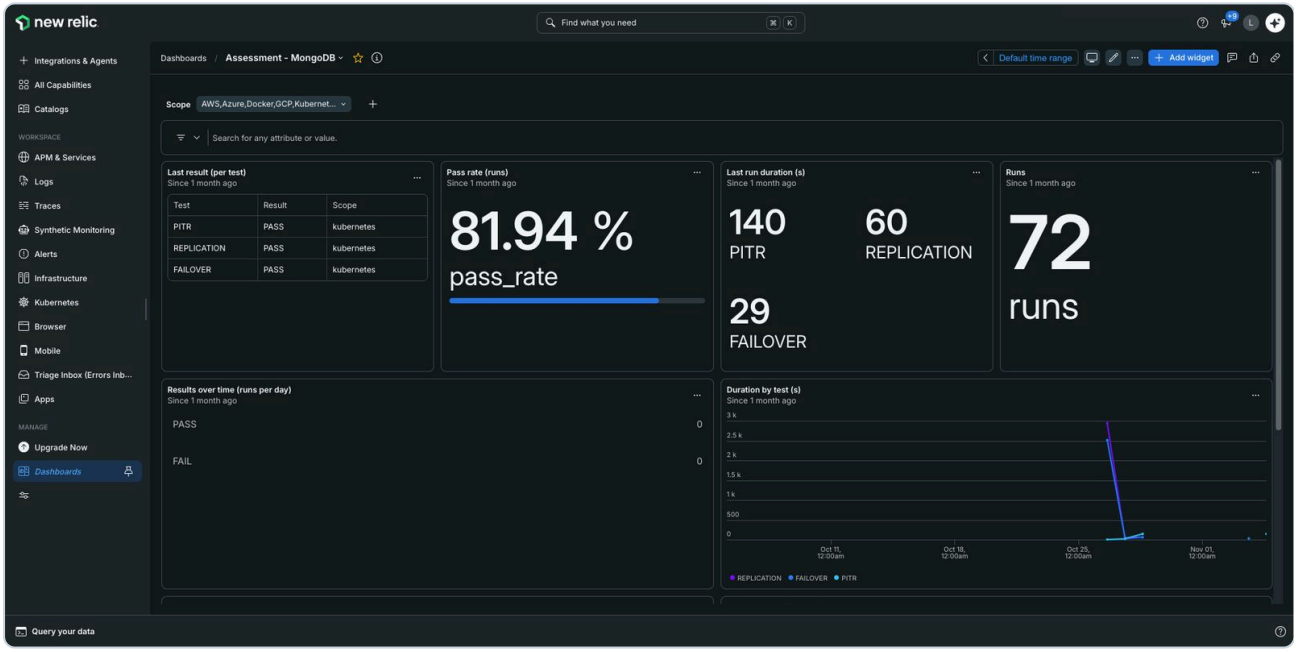


Figure 2.15 New Relic Assessment – MongoDB: 81.94% pass rate over 72 runs; latest PITR, replication and failover results

2.7 Staged IDURAR load: 10,000, 20,000 and 30,000 inserts and deletes

Each stage inserts business documents through the API and then deletes the same number. New

Relic shows each stage as a clear rise in operations.



Figure 2.16 Operations by type before the staged load



Figure 2.17 Stage 1: 10,000 inserts + 10,000 deletes

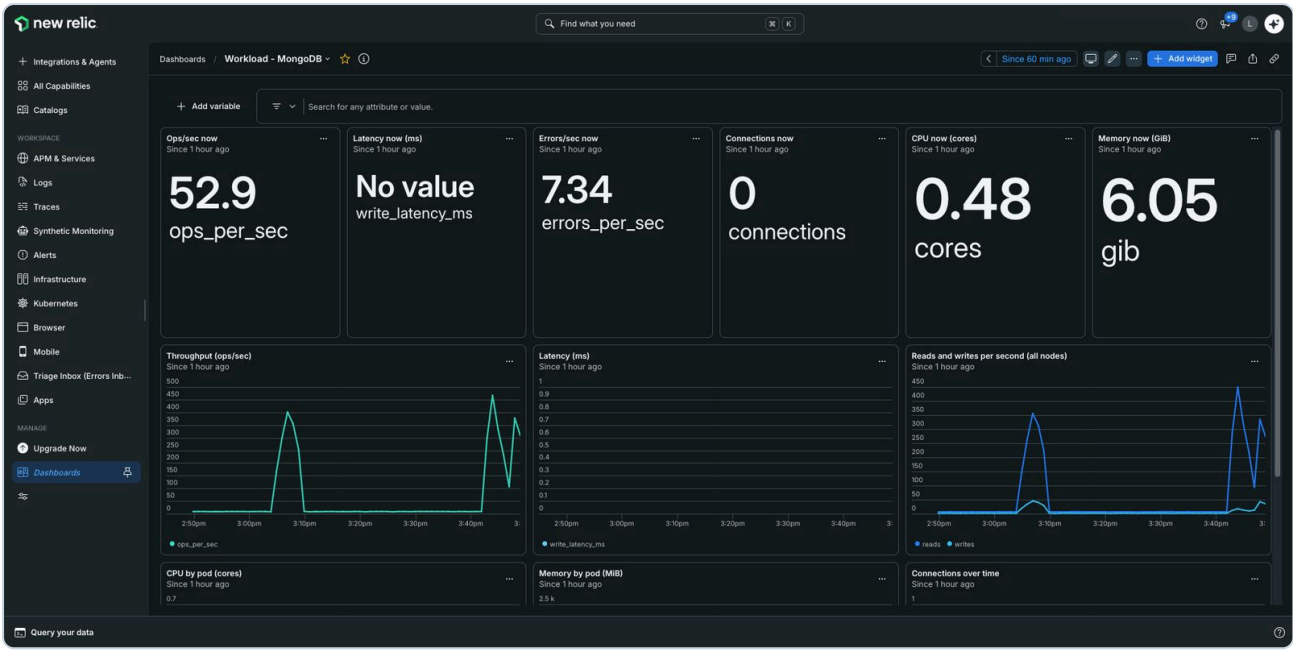


Figure 2.18 Workload dashboard during the staged load



Figure 2.19 Stage 2: 20,000 inserts + 20,000 deletes

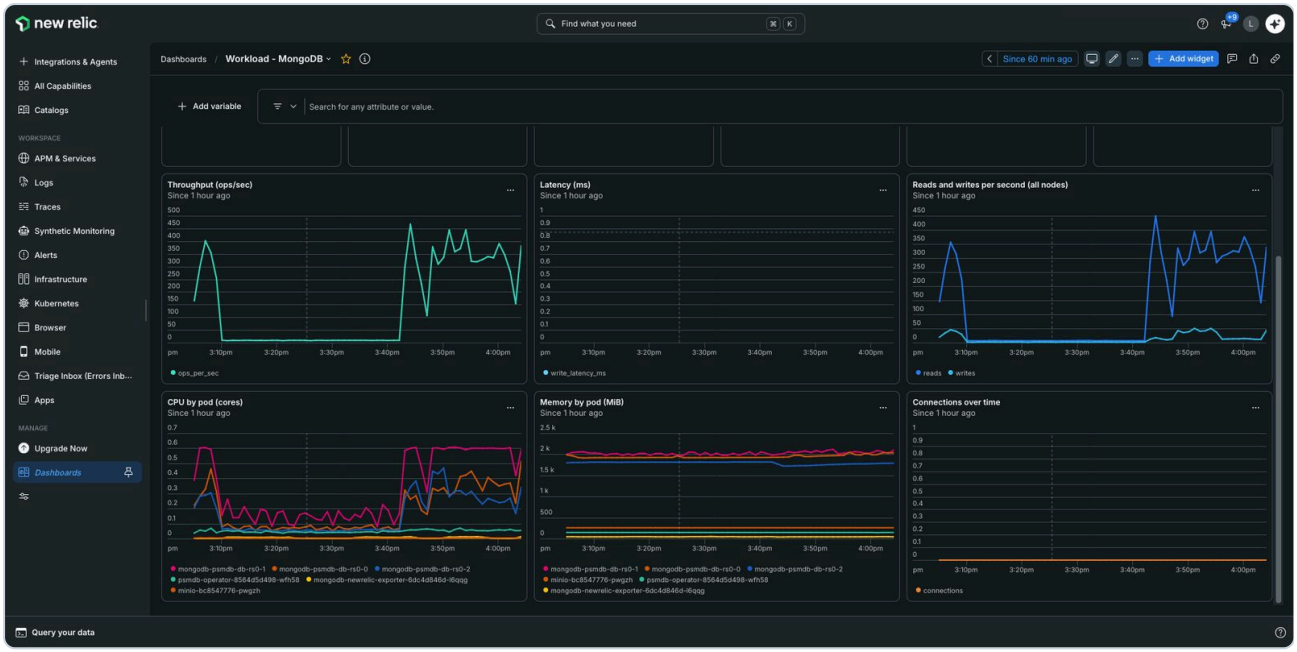


Figure 2.20 Stage 3: 30,000 inserts + 30,000 deletes

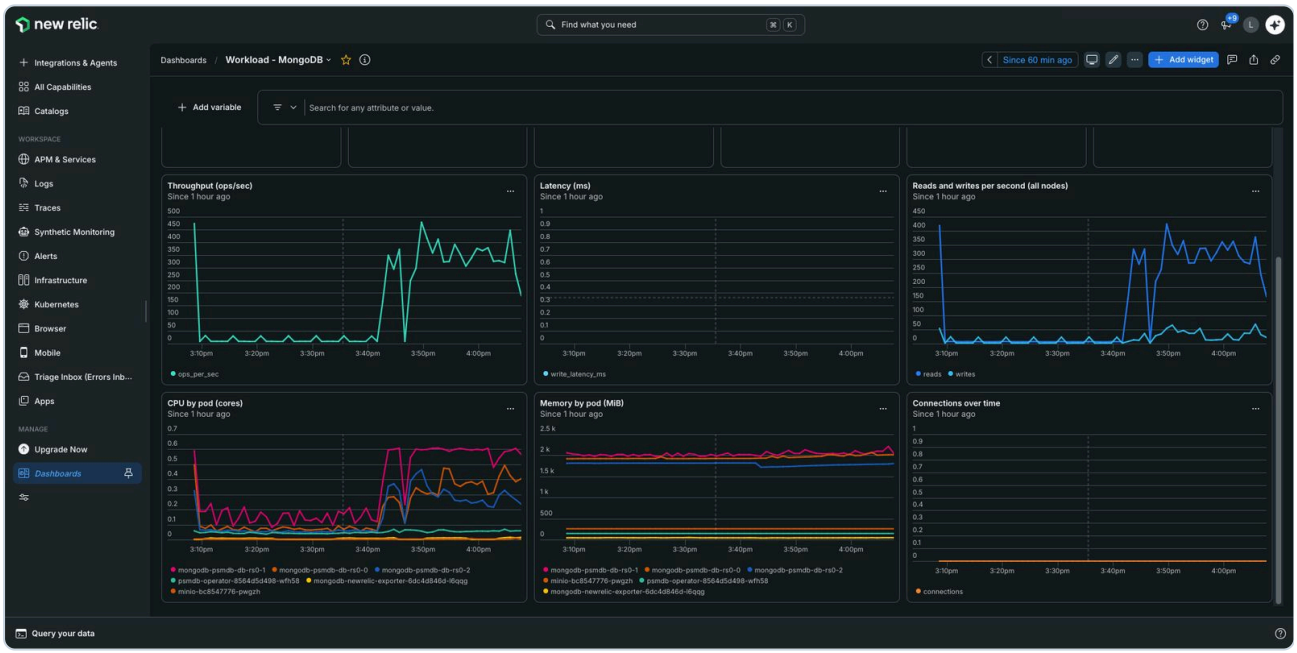


Figure 2.21 Replication test during the load



Figure 2.22 Operations by type across the stages

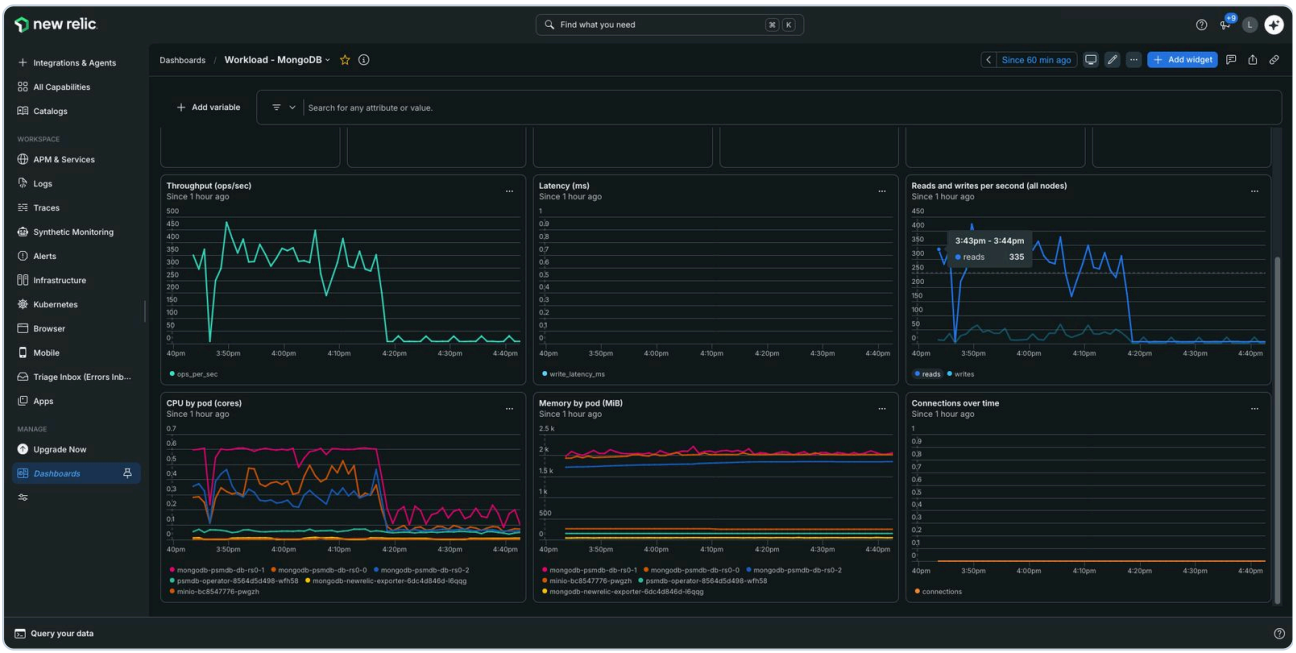


Figure 2.23 Final view



Figure 2.24 Operations by type after the run

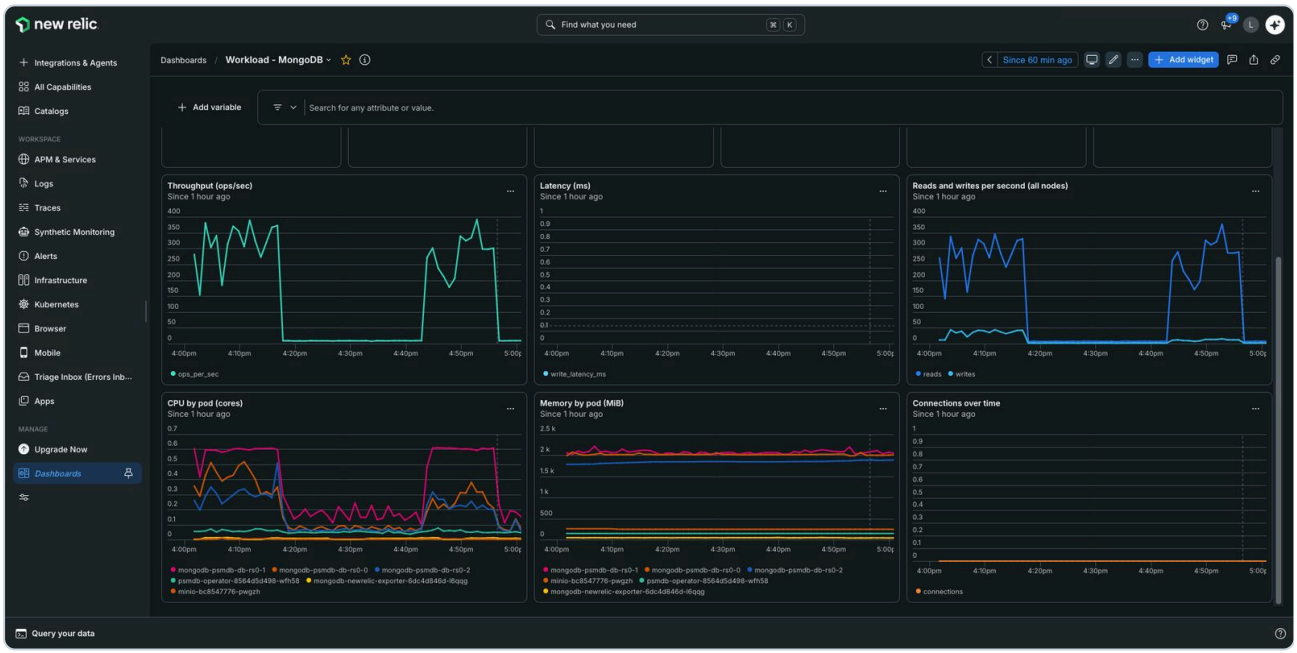


Figure 2.25 Workload – MongoDB after the run

OUTCOME

IDURAR's real API traffic is visible in New Relic, stage by stage, alongside RunShore's own workload and recovery tests.

03 Django on PostgreSQL

Python · PostgreSQL (CloudNativePG) · monitored with PMM

APPLICATION	DATABASE	CONNECTION	OBSERVABILITY	LOAD
Django (Python)	PostgreSQL · CloudNativePG	Read-write service via port-forward 5433	PMM	tools/client-apps/django/start-workload-django.sh

3.1 Start Django

```
bash tools/client-apps/runshore_apps.sh start django
```

Django admin opens at `127.0.0.1:8001/admin/`.

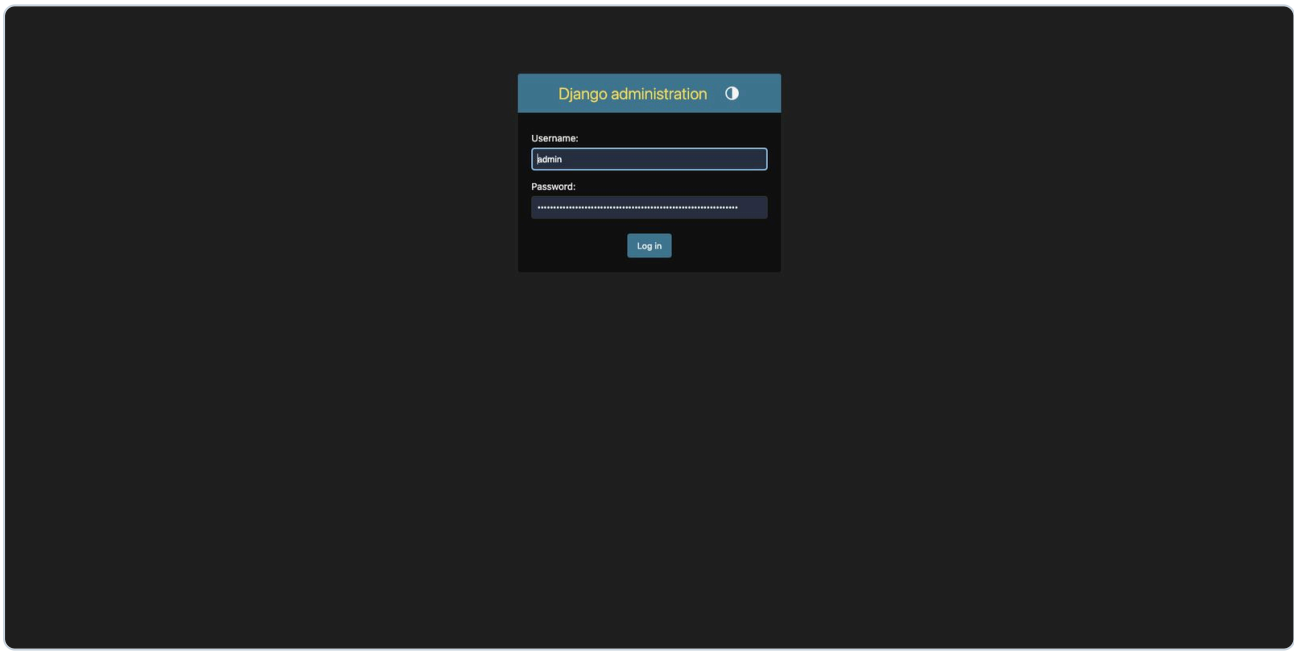


Figure 3.1 Django administration sign-in

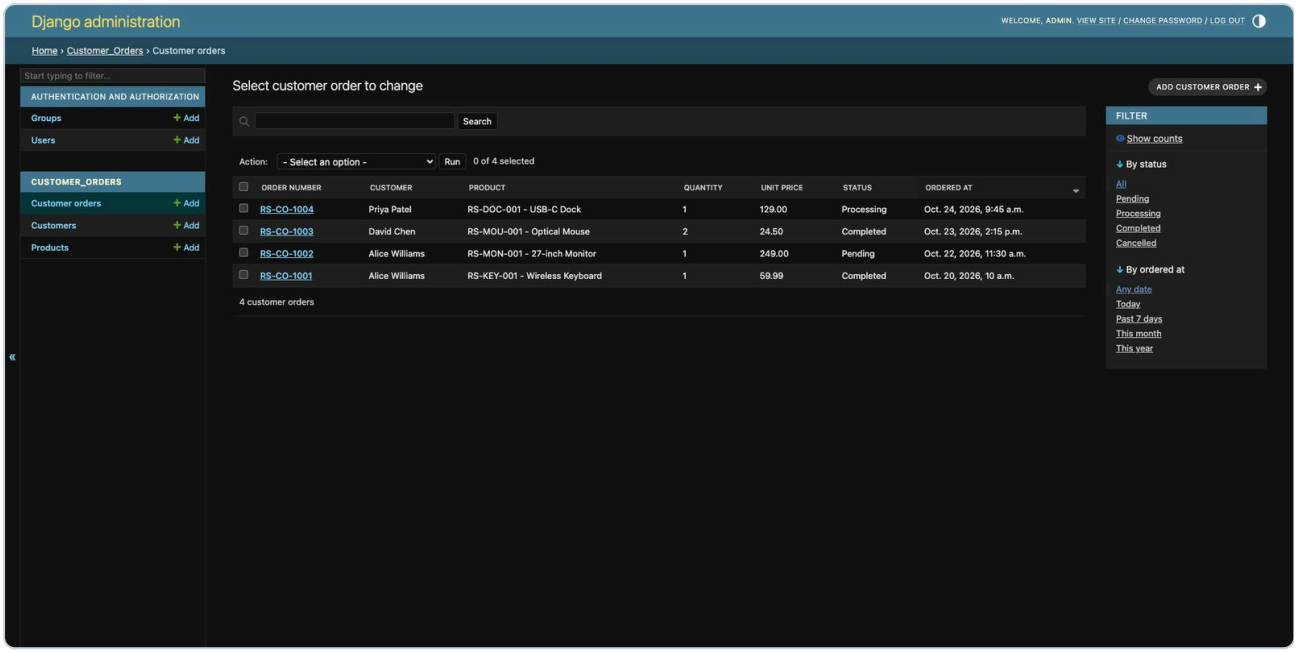


Figure 3.2 Customer orders stored in RunShore's PostgreSQL

3.2 Register PostgreSQL into PMM

```
bash tools/register-postgres-pmm.sh
```

- Finds the CloudNativePG cluster (`platform-postgres`) and the PMM pod.
- Checks for an existing registration first, so it is safe to run again.
- Registers the service and verifies it: PMM agent connected; node exporter, PostgreSQL exporter, pg_stat_statements agent and vmagent running.

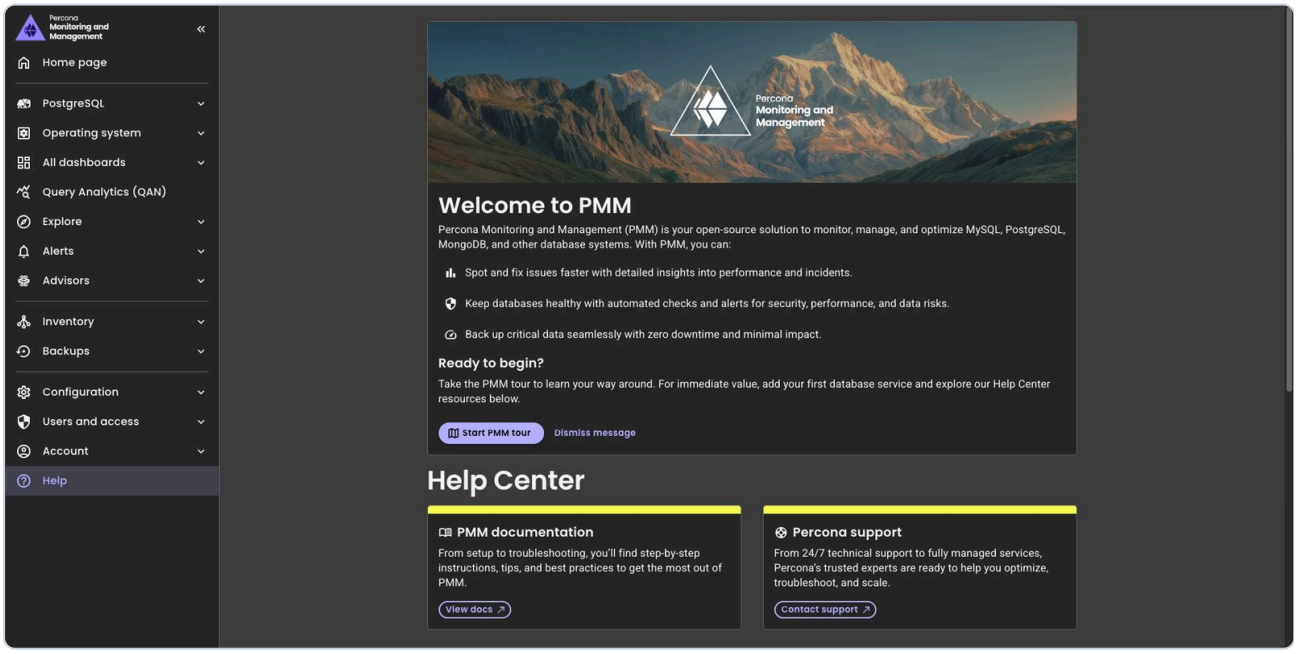


Figure 3.3 PMM home

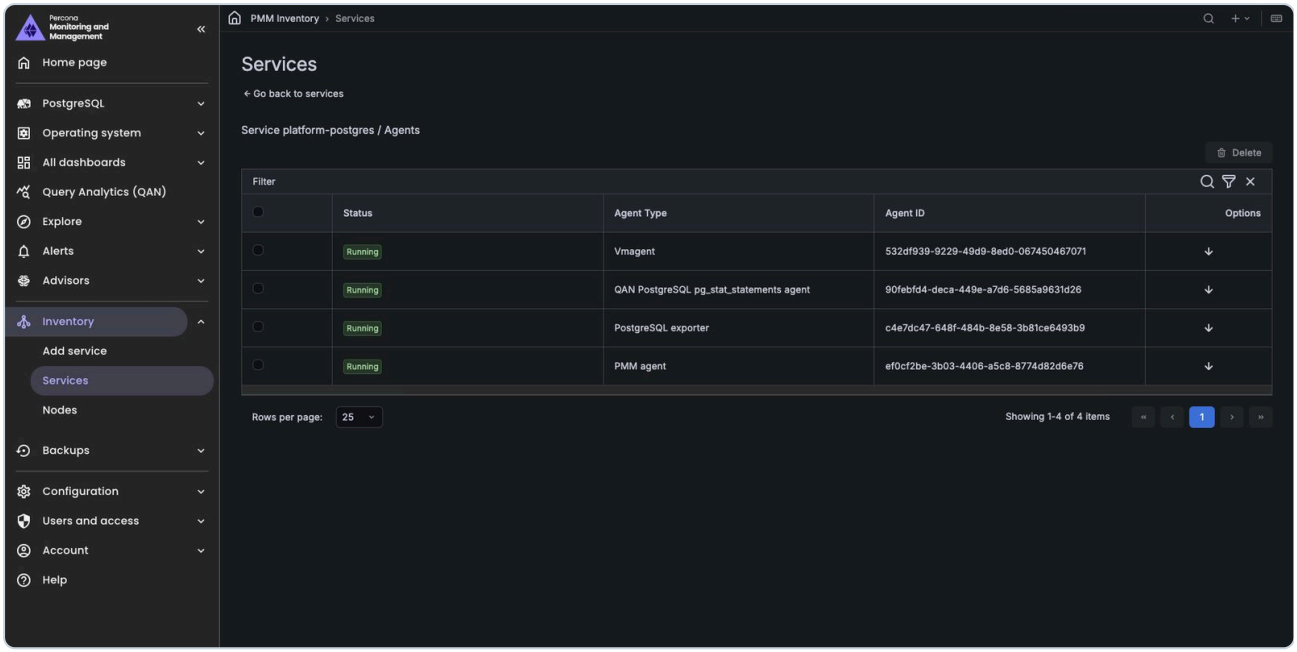


Figure 3.4 PMM Inventory: all four monitoring agents running for platform-postgres

3.3 Baseline

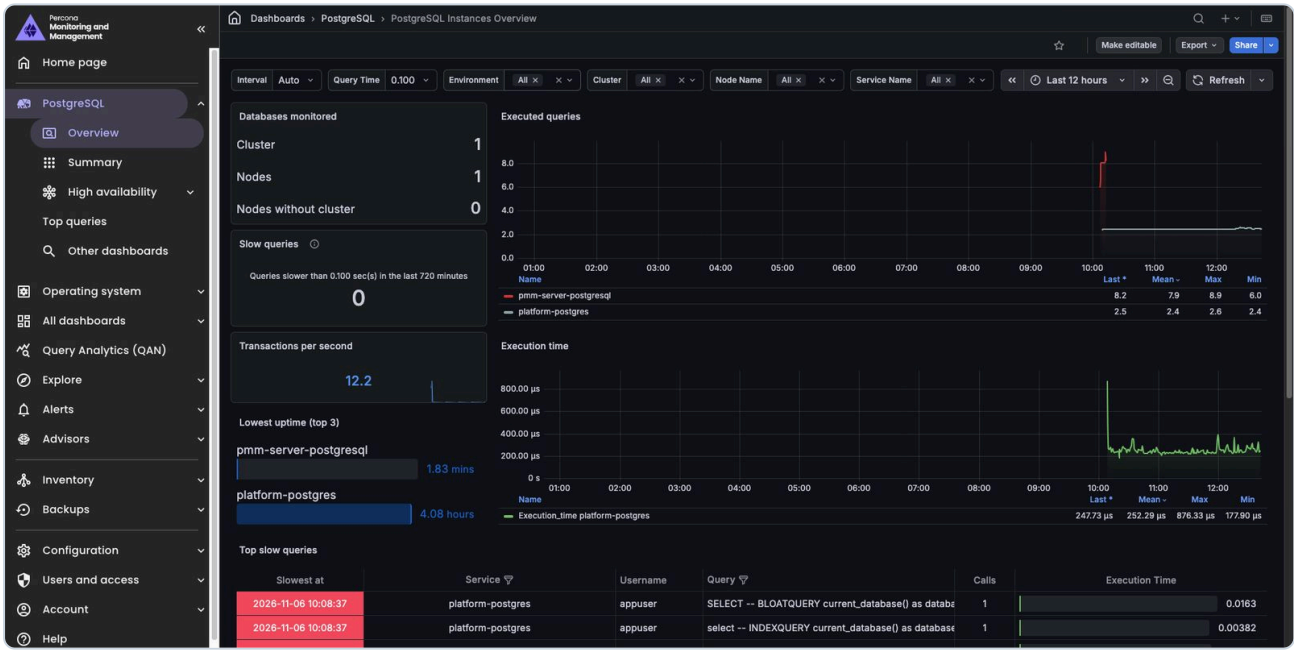


Figure 3.5 PMM PostgreSQL Instances Overview: baseline

3.4 Run the customer-order workload

```
tools/client-apps/django/start-workload-django.sh
```

```
RunShore Customer Order workload
Database: django
Cycles: 60
```

```
[START] Generating PostgreSQL activity.
[INFO] Completed 10/60 ... 60/60 cycles.
[PASS] Customer reads and aggregates executed.
[PASS] Temporary inserts and updates executed.
[PASS] Relationships verified.
[PASS] Transactions rolled back.
[PASS] Assessment data counts unchanged.
[PASS] PostgreSQL workload completed.
```

The workload's test transactions are rolled back, so the assessment data is left unchanged.

3.5 Observe in PMM

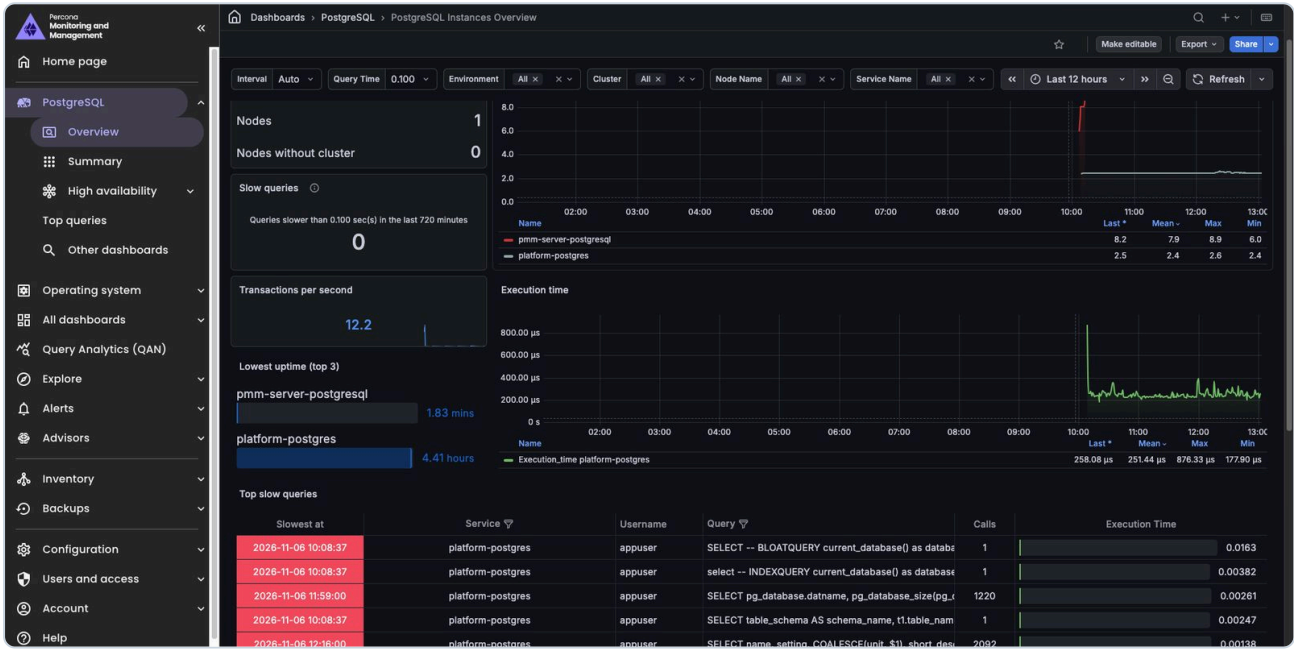


Figure 3.6 PostgreSQL Instances Overview after the workload

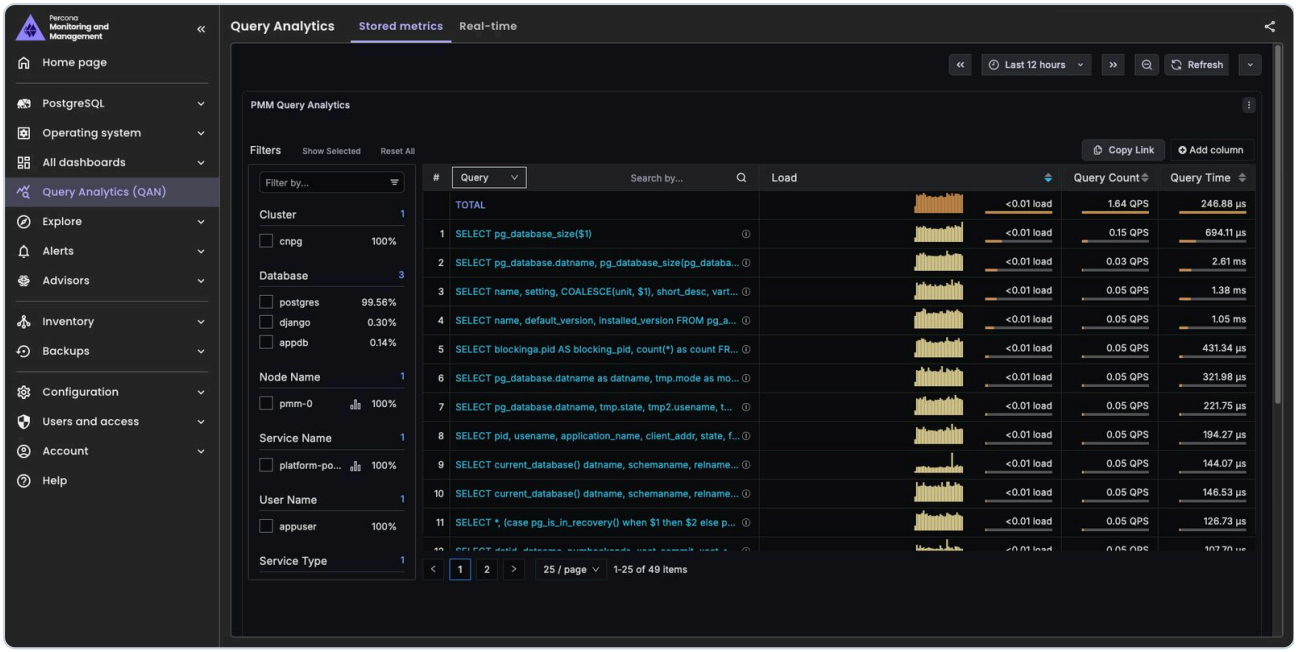


Figure 3.7 PMM Query Analytics: the queries behind the load

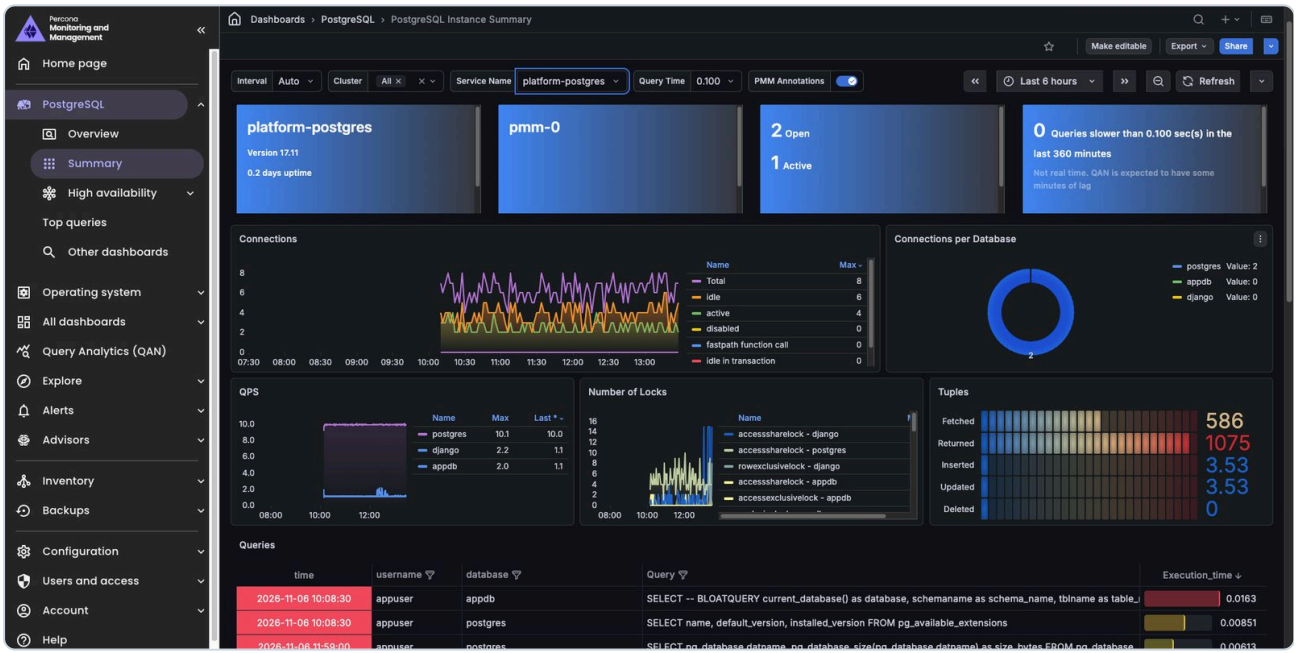


Figure 3.8 PostgreSQL Instance Summary: connections, locks and tuples

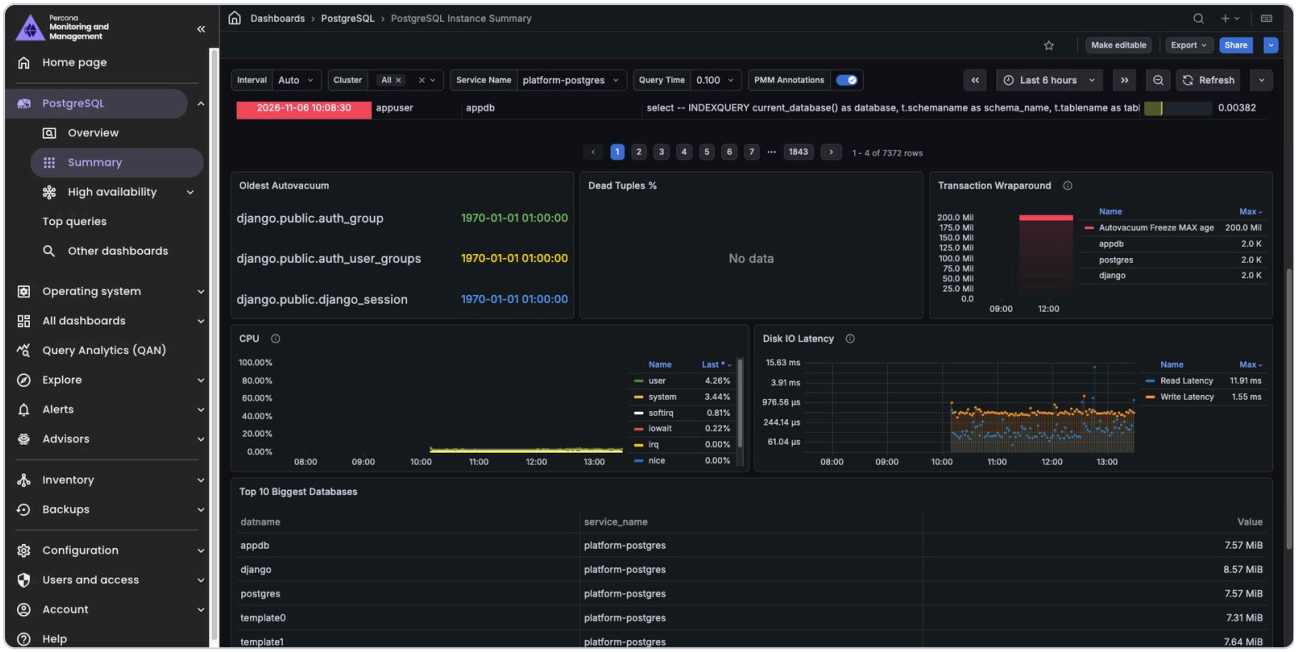


Figure 3.9 Instance Summary: oldest activity, CPU, disk latency and database sizes

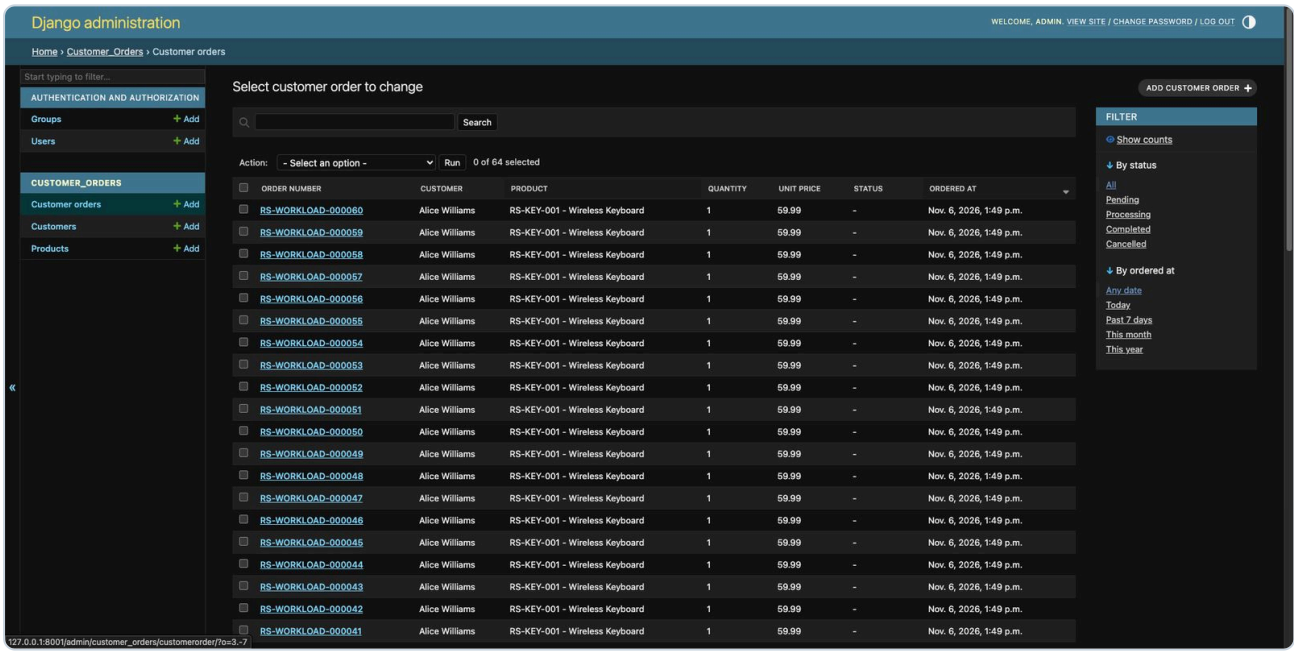


Figure 3.10 Increased customer orders in Django

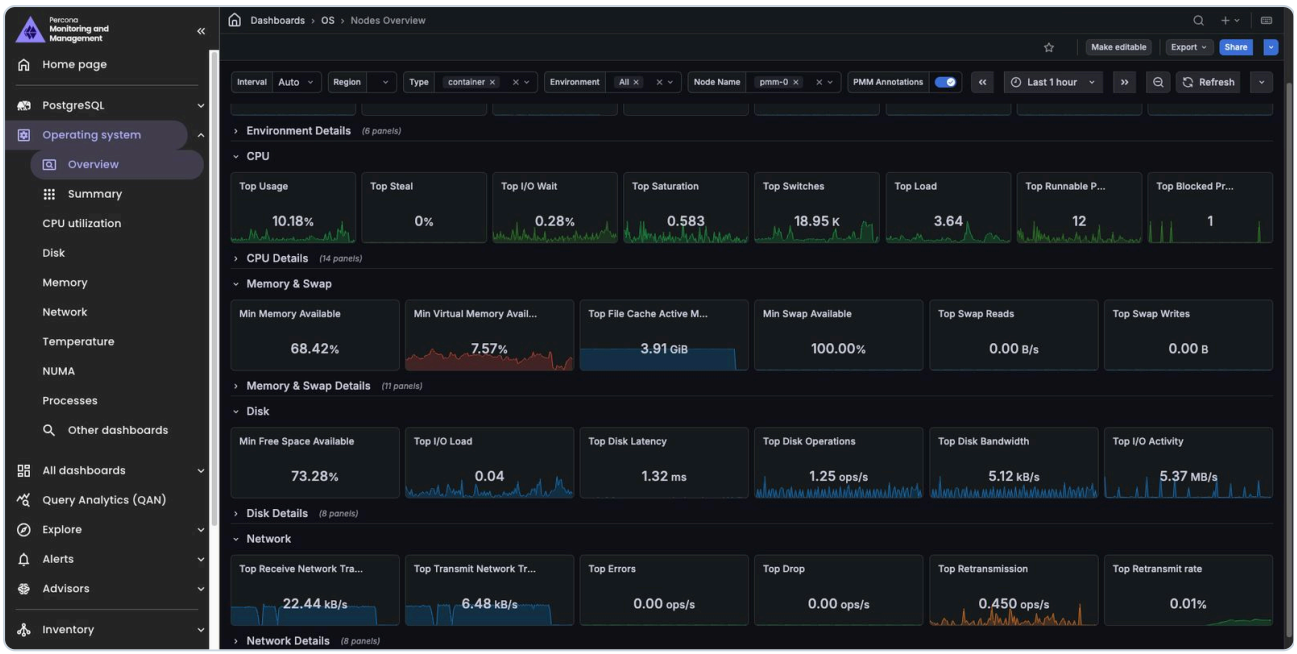


Figure 3.11 PMM Nodes Overview: CPU, memory, disk and network

OUTCOME

Django's customer orders run against RunShore's PostgreSQL, with every query visible in PMM.

04

Customer Orders (Spring Boot) on SAP ASE

Java · SAP ASE / Sybase · monitored with Splunk

APPLICATION	DATABASE	CONNECTION	OBSERVABILITY	LOAD
Customer Orders (Java Spring Boot)	SAP ASE / Sybase	Read-only (SELECT-only) application login	Splunk	Customer Orders workload

4.1 Baseline

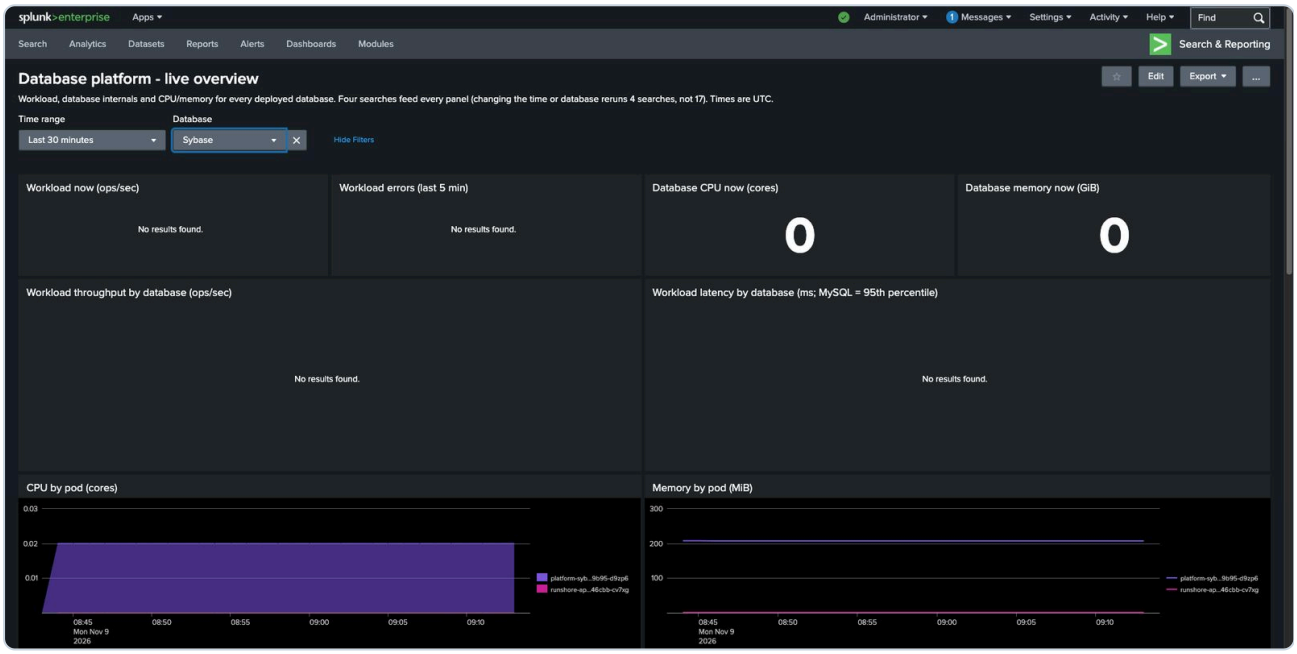


Figure 4.1 Splunk: database platform live overview for Sybase

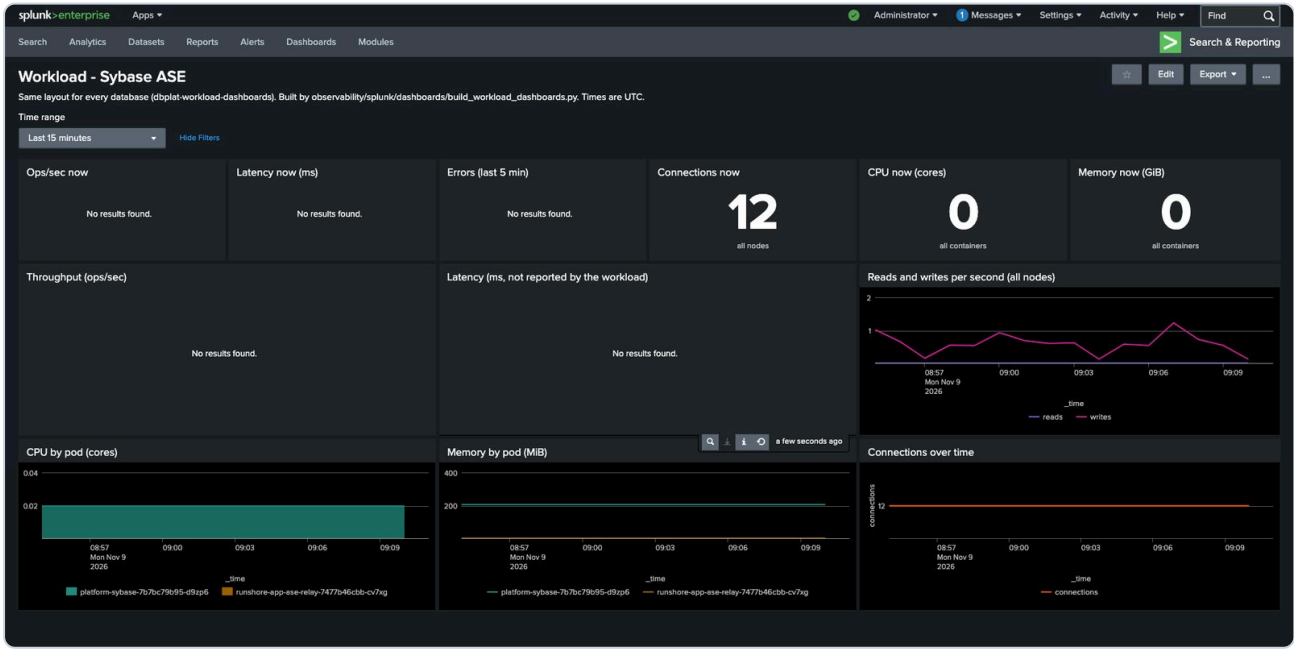


Figure 4.2 Splunk Workload – Sybase ASE: 12 connections, before the load

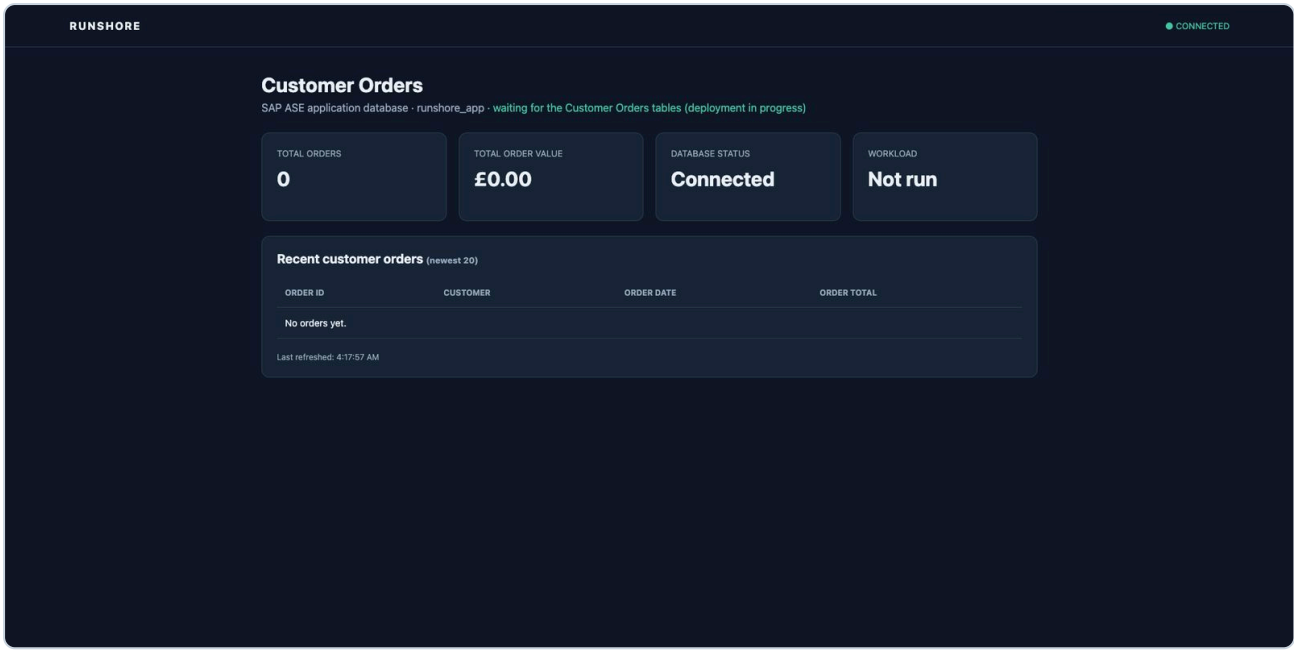


Figure 4.3 Customer Orders: connected, no orders, workload not run

4.2 Begin the load

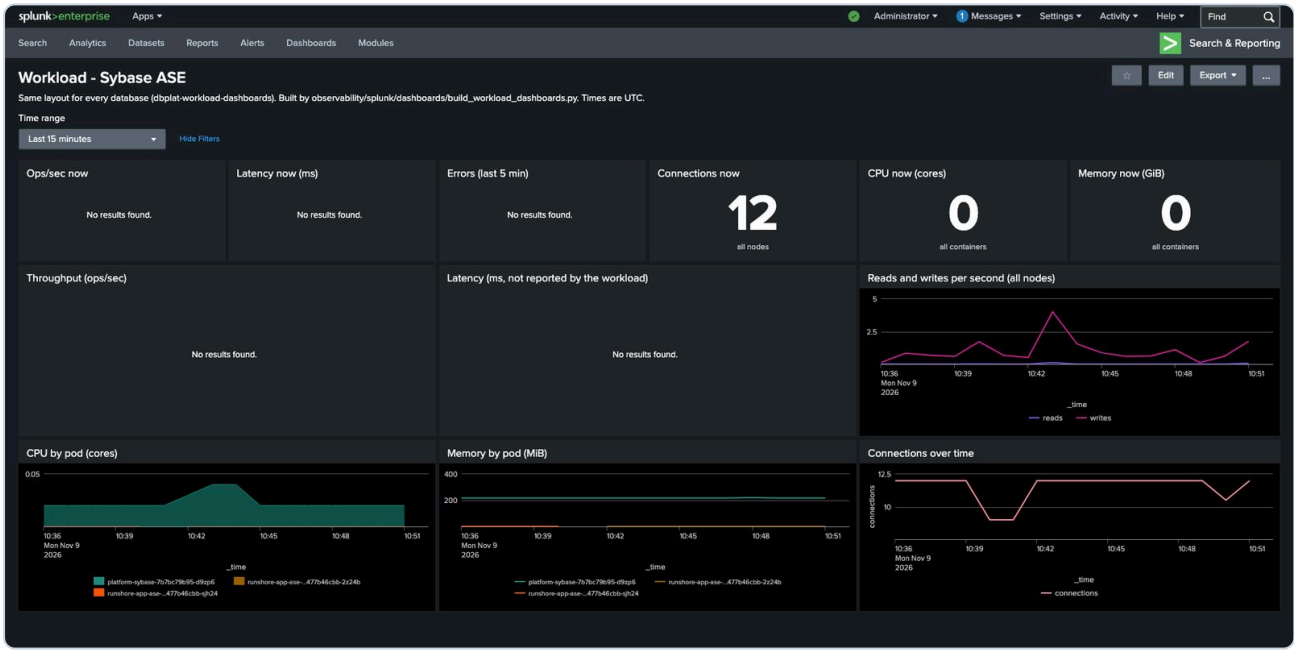


Figure 4.4 Splunk Workload – Sybase ASE during the load

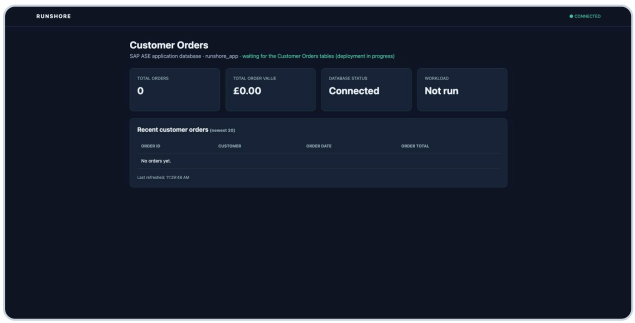


Figure 4.5 Not run

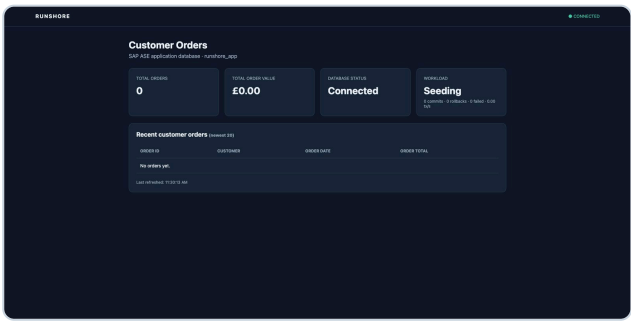


Figure 4.6 Seeding

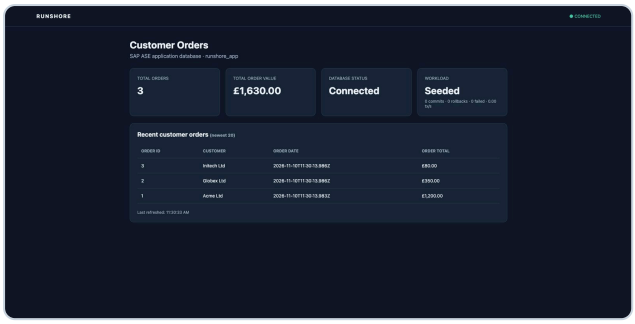


Figure 4.7 Seeded: 3 orders, £1,630.00

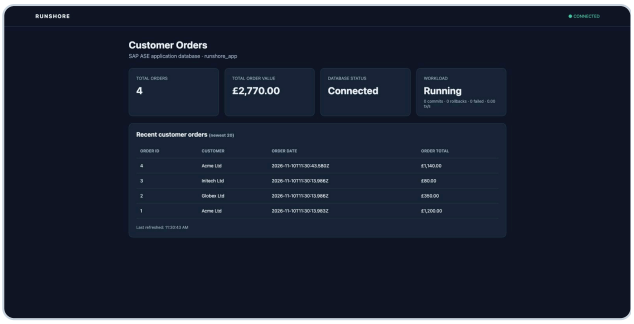


Figure 4.8 Running: 4 orders, £2,770.00

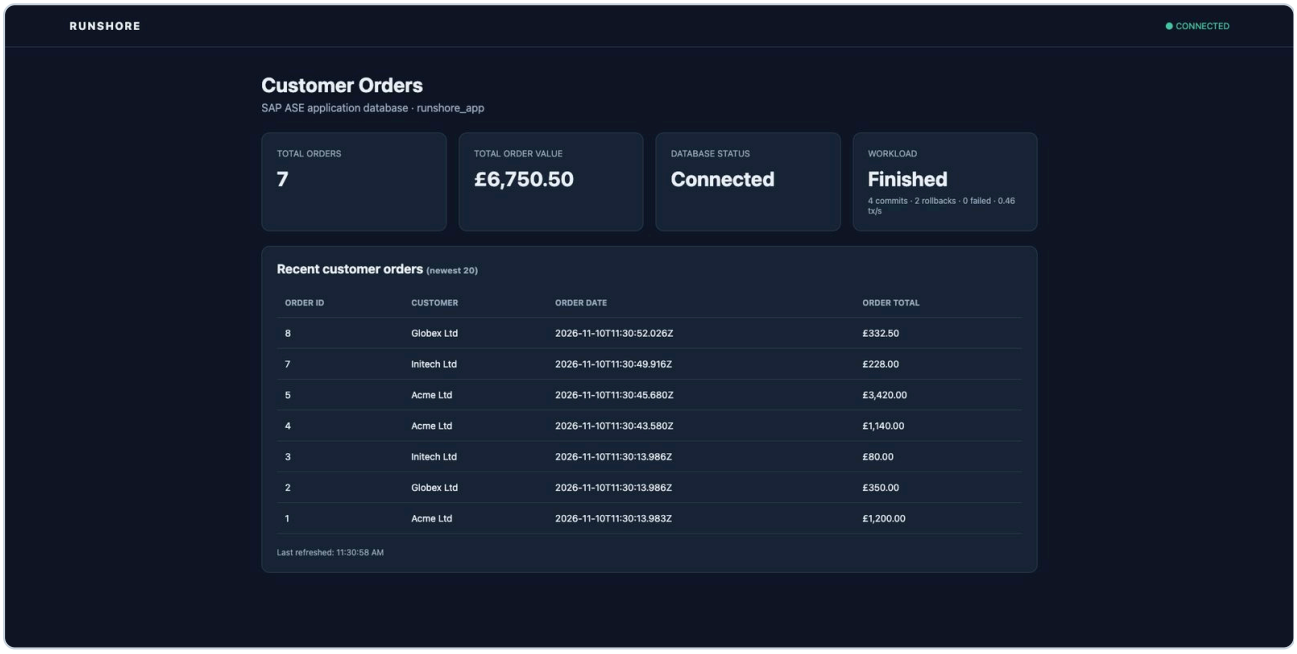


Figure 4.9 Finished: 7 orders, £6,750.50

OUTCOME

A Java application on SAP ASE: it reads orders through a read-only (SELECT-only) login while the workload adds orders, all observed in Splunk.

What this shows

- RunShore's databases serve real applications, not only synthetic benchmarks.
- Every database is paired with an observability tool, and the application's own traffic is visible there.
- Loads are tagged, staged and cleaned up, so every run starts from a clean environment.
- Recovery tests (PITR, failover, replication) run while applications are connected.

RunShore · Database assessment and observability platform